



Université Moulay Ismail
Faculté des Sciences Meknès
Département de Mathématiques et Informatique

Réseaux de Neurones Artificiels

Capacité d'Approximation et Méthodes d'Apprentissage

Rapport du projet de fin d'études

Présenté par :

Tafoughalti Youssef

Encadré par :

Pr. El Mehdi Amhraoui

Membres du jury :

Pr. Abdelilah monir

Pr. Salim El Azami El-Idrissi

Année Universitaire : 2024-2025

Table des matières

1	Introduction	3
1.1	Contexte et motivation	3
1.2	Objectifs du rapport	3
2	Fondements des Réseaux de Neurones Artificiels	5
2.1	Modèle paramétrique et neurone formel	5
2.1.1	Le neurone formel de McCulloch et Pitts	5
2.1.2	Définition du perceptron simple	5
2.1.3	Neurone AND	6
2.1.4	Neurone OR	7
2.1.5	Neurone NOT	7
2.1.6	Neurone XOR	7
2.2	Réseaux de Neurones Artificiels et Architectures	10
2.2.1	Réseaux Feedforward (FFNN)	10
2.2.2	Réseaux Neuronaux Récurrents (RNN)	10
2.2.3	Perceptron Multicouche (MLP)	11
2.3	Fonctions d'activation	13
2.3.1	Sigmoïde	13
2.3.2	ReLU (Rectified Linear Unit)	13
2.3.3	Tanh (Tangente hyperbolique)	14
2.3.4	Comparaison des fonctions d'activation	14
3	Théorèmes sur la Capacité d'Approximation des RNAs	15
3.1	Théorème d'Approximation Universelle	15
3.2	Théorème de Cybenko	16
3.2.1	Preliminaires et théorème	16
3.2.2	Théorème	17
3.2.3	Preuve du théoreme de Cybenko	19
3.2.4	Limites du théorème de Cybenko	20
3.3	Théorème de Hornik : Extension aux fonctions continues	21
3.3.1	Énoncé rigoureux du théorème	21
3.3.2	Preuve du théoreme de Hornik	22
3.4	Comparaison des résultats et implications pratiques	23
3.4.1	Conséquences mathématiques	24
3.4.2	Implications pratiques	24
3.4.3	Portée et limites en pratique	24

3.4.4	Conclusion du chapitre	24
4	Méthodes d'Apprentissage des Réseaux de Neurones	25
4.1	Notions fondamentales de l'apprentissage	25
4.1.1	Objectifs de l'apprentissage	25
4.1.2	Types d'apprentissage	26
4.2	Optimisation et apprentissage supervisé d'un neurone formel	27
4.2.1	Formulation du problème	27
4.2.2	Problème de régression	27
4.2.3	Problème de classification	29
4.2.4	Fonctions de coût classiques	32
4.2.5	Formulation et optimisation	33
4.2.6	Remarques	34
4.2.7	Algorithmes d'optimisation	34
5	Les réseaux de neurones convolutifs	42
5.1	Introduction sur les Convolutional Neural Networks	42
5.2	Architecture générale d'un CNN	43
5.2.1	Convolution	43
5.2.2	Pooling	44
5.2.3	Flatten	45
5.2.4	Fully Connected	46
5.2.5	SoftMax	46
5.3	Entraînement du réseau de neurones convolutifs	46
5.3.1	La propagation avant (Forward Propagation) :	47
5.3.2	Le calcul de la perte (Loss Calculation) :	47
5.3.3	La rétropropagation et l'optimisation (Backpropagation Optimiza- tion) :	47
5.4	Application des modèles ANN et CNN	47

Chapitre 1

Introduction

1.1 Contexte et motivation

Les **réseaux de neurones artificiels** (RNAs) ont révolutionné le domaine de l'intelligence artificielle et de l'apprentissage automatique. Inspirés du fonctionnement biologique du cerveau humain, ils permettent d'apprendre des relations complexes à partir de données, avec des applications allant de la reconnaissance d'images au traitement du langage naturel.

L'un des aspects fondamentaux des RNAs est leur capacité à **approximer des fonctions complexes**, ce qui en fait un outil puissant pour de nombreux domaines scientifiques et industriels. Cependant, bien que ces modèles soient performants, leur efficacité dépend de plusieurs facteurs :

- le choix de l'architecture du réseau (nombre de couches et de neurones) ;
- la sélection des fonctions d'activation ;
- la méthode d'apprentissage et d'optimisation utilisée ;
- les défis liés à l'entraînement, tels que le sur-apprentissage et le problème du gradient évanescent.

Dans ce contexte, ce rapport vise à étudier les **propriétés d'approximation des RNAs** ainsi que les différentes **méthodes d'apprentissage**.

1.2 Objectifs du rapport

L'objectif principal de ce rapport est d'étudier les **réseaux de neurones artificiels** (RNAs) sous l'angle de leur **capacité d'approximation** et des **méthodes d'apprentissage**. Pour cela, plusieurs axes seront explorés :

1. Comprendre la structure des réseaux de neurones artificiels

Nous présenterons les notions fondamentales des RNAs, notamment les concepts de **neurone formel**, de **réseau de neurones feedforward** et de **perceptron multicouche**, ainsi que les différentes **fonctions d'activation** utilisées en pratique.

2. Étudier les théorèmes d'approximation universelle

Nous analyserons les résultats théoriques majeurs qui garantissent que les RNAs

peuvent approximer toute fonction continue sous certaines conditions. En particulier, nous discuterons des théorèmes de **Cybenko**, **Hornik** , en illustrant leurs implications.

3. **Explorer les méthodes d'apprentissage des RNAs**

Nous détaillerons les techniques d'apprentissage supervisé, en mettant l'accent sur la **descente de gradient**, l'algorithme de **rétropropagation** et les différentes **fonctions de perte** comme l'erreur quadratique moyenne (MSE) et l'entropie croisée (cross-entropy).

Fondements des Réseaux de Neurones Artificiels

2.1 Modèle paramétrique et neurone formel

Un modèle paramétrique est un modèle mathématique où la relation entre les entrées et les sorties est entièrement déterminée par un ensemble fini de paramètres. Les réseaux de neurones artificiels (RNAs) appartiennent à cette catégorie, car ils utilisent des poids et des biais pour ajuster leur comportement.

2.1.1 Le neurone formel de McCulloch et Pitts

Le premier modèle de neurone artificiel a été proposé par **McCulloch et Pitts** en 1943. Il repose sur une combinaison linéaire des entrées suivie d'une fonction d'activation.

Un neurone artificiel reçoit une entrée $\mathbf{x} = (x_1, x_2, \dots, x_n)$ pondérées par des poids $\mathbf{w} = (w_1, w_2, \dots, w_n)$. La sortie est donnée par l'équation suivante :

$$z = \sum_{i=1}^n w_i x_i + b \quad (2.1)$$

où :

- x_i sont les entrées,
- w_i sont les poids synaptiques,
- b est le biais.

La sortie du neurone est obtenue en appliquant une fonction d'activation f :

$$y = f(z) \quad (2.2)$$

2.1.2 Définition du perceptron simple

Un **perceptron simple** est un modèle de neurone artificiel utilisé pour la classification binaire. Il prend en entrée un vecteur de caractéristiques $\mathbf{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$ et calcule une sortie y selon la formule suivante :

$$y = \varphi \left(\sum_{i=1}^n w_i x_i + b \right) = \varphi(\mathbf{w} \cdot \mathbf{x} + b)$$

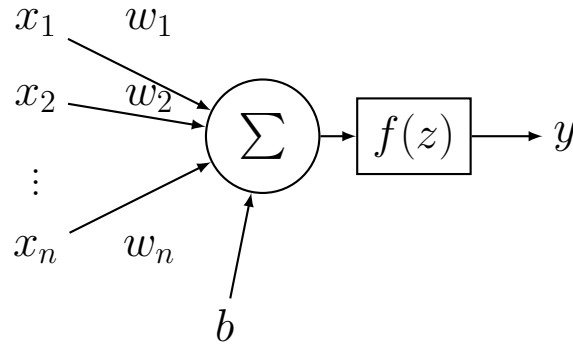


FIGURE 2.1 – Modèle d'un neurone formel

où :

- $\mathbf{w} = (w_1, w_2, \dots, w_n)$ est le vecteur des poids synaptiques,
- $b \in \mathbb{R}$ est le biais (ou seuil),
- φ est une fonction d'activation, typiquement la fonction échelon :

$$\varphi(z) = \begin{cases} 1 & \text{si } z \geq 0 \\ 0 & \text{sinon} \end{cases}$$

Après avoir défini le perceptron simple, il est utile d'illustrer concrètement son fonctionnement. Pour cela, nous allons l'appliquer à des fonctions logiques de base (AND, OR, ...), qui constituent des exemples simples mais pertinents. Ces fonctions, avec leurs entrées binaires, permettent de mieux comprendre comment un perceptron prend une décision à partir d'une combinaison linéaire suivie d'un seuil..

2.1.3 Neurone AND

Le neurone AND est un perceptron simple avec deux entrées. Il produit une sortie de 1 seulement lorsque les deux entrées sont égales à 1. Son architecture est la suivante :

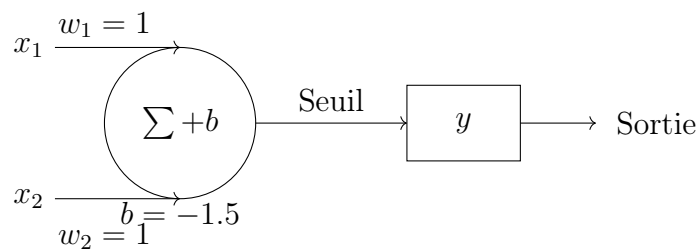


FIGURE 2.2 – Neurone AND

x_1	x_2	$x_1 w_1 + x_2 w_2 + b$	Somme	Seuil (step)	y
0	0	$0 + 0 - 1.5$	-1.5	0	0
0	1	$0 + 1 - 1.5$	-0.5	0	0
1	0	$1 + 0 - 1.5$	-0.5	0	0
1	1	$1 + 1 - 1.5$	0.5	1	1

TABLE 2.1 – Table de vérité du neurone AND avec $w_1 = w_2 = 1$ et $b = -1.5$

2.1.4 Neurone OR

Le neurone OR est un perceptron simple qui prend également deux entrées et produit une sortie de 1 si au moins une des entrées est égale à 1. Son architecture est la suivante :

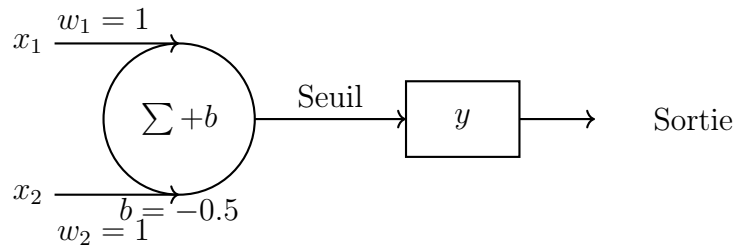


FIGURE 2.3 – Neurone OR

x_1	x_2	$x_1w_1 + x_2w_2 + b$	Somme	Seuil (step)	y
0	0	$0 + 0 - 0.5$	-0.5	0	0
0	1	$0 + 1 - 0.5$	0.5	1	1
1	0	$1 + 0 - 0.5$	0.5	1	1
1	1	$1 + 1 - 0.5$	1.5	1	1

TABLE 2.2 – Table de vérité du neurone OR avec $w_1 = w_2 = 1$ et $b = -0.5$

2.1.5 Neurone NOT

Le neurone NOT est un perceptron simple avec une seule entrée et produit l'inverse de cette entrée. Son architecture est la suivante :

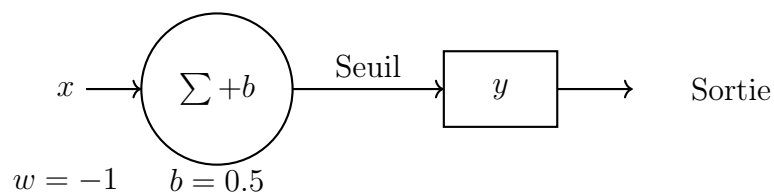


FIGURE 2.4 – Neurone NOT

x	$xw + b$	Somme	Seuil (step)	y
0	$0 \times (-1) + 0.5$	0.5	1	1
1	$1 \times (-1) + 0.5$	-0.5	0	0

TABLE 2.3 – Table de vérité du neurone NOT avec $w = -1$ et $b = 0.5$

2.1.6 Neurone XOR

Un perceptron simple, est capable de modéliser certaines fonctions logiques telles que AND, OR ou NOT. Ces fonctions sont dites **linéairement séparables**, ce qui signifie

qu'il existe une droite (ou un hyperplan en dimension supérieure) qui permet de séparer les cas où la sortie vaut 1 de ceux où elle vaut 0.

Naturellement, on peut se demander : *peut-on approximer n'importe quelle fonction logique à l'aide d'un seul perceptron simple ?*

La réponse est **non**.

Preuve :

On suppose qu'il existe un perceptron simple qui vérifie le "ou exclusif" (XOR).

Soit

$$F(x, y) = H(ax + by + c) \quad \text{où} \quad H(z) = \begin{cases} 1 & \text{si } z \geq 0 \\ 0 & \text{sinon} \end{cases}$$

On a :

$$F(1, 0) = 1 = H(a + c) \Rightarrow a + c \geq 0$$

$$F(0, 1) = 1 = H(b + c) \Rightarrow b + c \geq 0$$

$$F(0, 0) = 0 = H(c) \Rightarrow c < 0$$

$$F(1, 1) = 0 = H(a + b + c) \Rightarrow a + b + c < 0$$

Donc :

$$\begin{cases} a + c \geq 0 \\ b + c \geq 0 \\ c < 0 \end{cases} \Rightarrow \begin{cases} a \geq -c \\ b \geq -c \end{cases} \Rightarrow a + b \geq -2c$$

Or :

$$a + b + c \geq -2c + c = -c > 0$$

Ce qui contredit le fait que :

$$F(1, 1) = 0 \Rightarrow a + b + c < 0$$

Conclusion : Il y a contradiction. Donc, un perceptron simple ne peut pas modéliser la fonction "ou exclusif" (XOR).

Un contre-exemple fondamental est donné par la fonction XOR, définie par la table de vérité suivante :

x_1	x_2	XOR(x_1, x_2)
0	0	0
0	1	1
1	0	1
1	1	0

Si l'on représente ces points dans le plan, les sorties égales à 1 (les points (0, 1) et (1, 0)) sont situées sur une diagonale, tandis que les sorties égales à 0 ((0, 0) et (1, 1)) sont sur l'autre diagonale. **Il est donc impossible de les séparer par une seule droite .**

La fonction XOR nécessite une **architecture ou moins à deux perceptrons** car cette fonction n'est pas linéairement séparable. L'architecture suivante montre un réseau avec une couche cachée.

- Une couche cachée composée de deux perceptrons : l'un détectant $x_1 \wedge \neg x_2$, l'autre $\neg x_1 \wedge x_2$;
- Une couche de sortie réalisant la conjonction (AND).

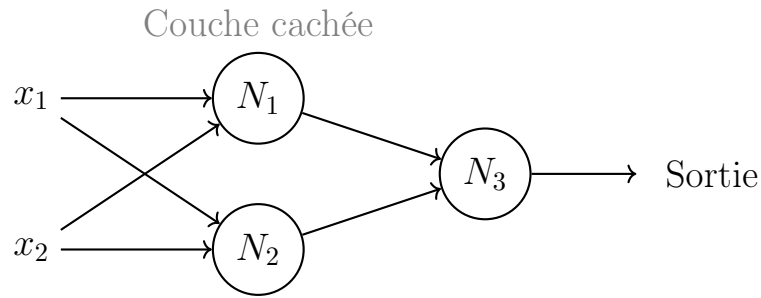


FIGURE 2.5 – XOR : architecture

x_1	x_2	$N_1 = \text{NAND}(x_1, x_2)$	$N_2 = \text{OR}(x_1, x_2)$	$N_3 = \text{AND}(N_1, N_2)$	$y \text{ (XOR)}$
0	0	1	0	0	0
0	1	1	1	1	1
1	0	1	1	1	1
1	1	0	1	0	0

TABLE 2.4 – Table de vérité du neurone XOR

Neurone	Fonction	Poids	Biais
N_1	NAND	$w_{11} = -2, w_{12} = -2$	$b_1 = 3$
N_2	OR	$w_{21} = 2, w_{22} = 2$	$b_2 = -1$
N_3	AND	$w_{31} = 2, w_{32} = 2$	$b_3 = -3$

TABLE 2.5 – Poids et biais pour la réalisation du XOR

$$f(z) = \begin{cases} 1 & \text{si } z > 0 \\ 0 & \text{sinon} \end{cases}$$

$$\begin{cases} N_1 & = f(-2x_1 - 2x_2 + 3) \\ N_2 & = f(2x_1 + 2x_2 - 1) \\ y = N_3 & = f(2N_1 + 2N_2 - 3) \end{cases}$$

2.2 Réseaux de Neurones Artificiels et Architectures

Un **réseau de neurones artificiel (RNA)** est un modèle mathématique conçu pour simuler le comportement des réseaux de neurones biologiques. Il s'agit d'un système **composé de neurones artificiels interconnectés**, organisés en couches, qui traite de l'information en réalisant des opérations élémentaires. Chaque neurone reçoit des signaux, les transforme à l'aide d'une **fonction d'activation**, puis transmet le résultat aux neurones suivants.

Les réseaux de neurones artificiels peuvent être classés en différentes architectures, en fonction de la manière dont les neurones sont organisés et connectés. Voici un aperçu des architectures les plus courantes :

2.2.1 Réseaux Feedforward (FFNN)

Un réseau de neurones **feedforward** (ou réseau de neurones à propagation avant) est l'architecture la plus simple. Les données circulent dans une seule direction, de l'entrée vers la sortie, sans boucle. Il est généralement constitué de trois types de couches :

- **Couche d'entrée** : reçoit les données brutes.
- **Couches cachées** : traitent les informations pour extraire des caractéristiques.
- **Couche de sortie** : donne la prédiction ou le résultat final.

L'architecture des réseaux feedforward est utilisée dans des tâches de classification ou de régression simples, où les données sont indépendantes et identiquement distribuées.

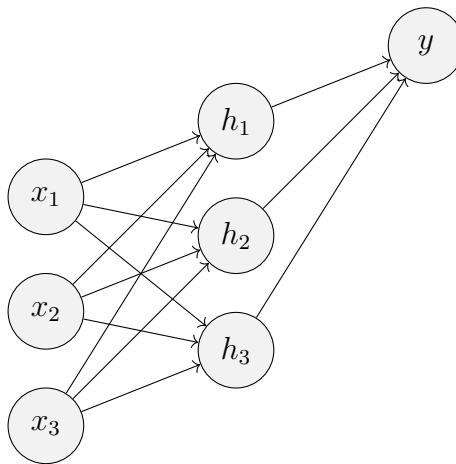
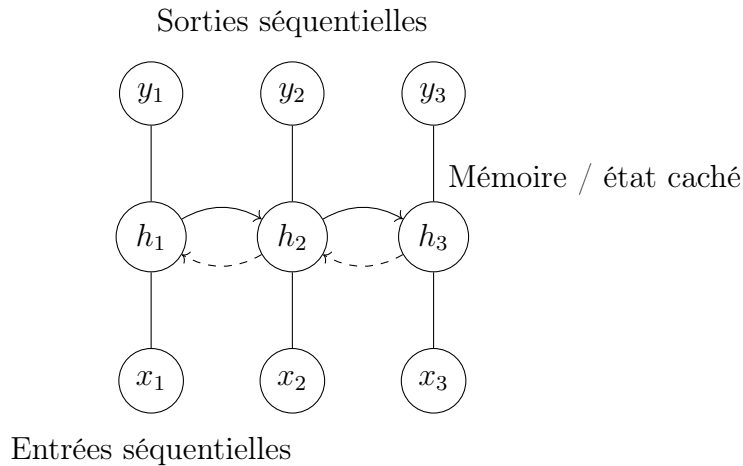


FIGURE 2.6 – Architecture d'un réseau Feedforward (FFNN)

2.2.2 Réseaux Neuronaux Récurrents (RNN)

Les **Réseaux Neuronaux Récurrents (RNN)** sont adaptés au traitement des données séquentielles. Contrairement aux réseaux feedforward, les RNN contiennent des connexions qui forment des boucles, permettant au réseau de "se souvenir" d'informations passées. Cela les rend efficaces pour des tâches telles que le traitement du langage naturel ou les séries temporelles.

Schéma d'un Réseau Neuronal Récurrent (RNN)

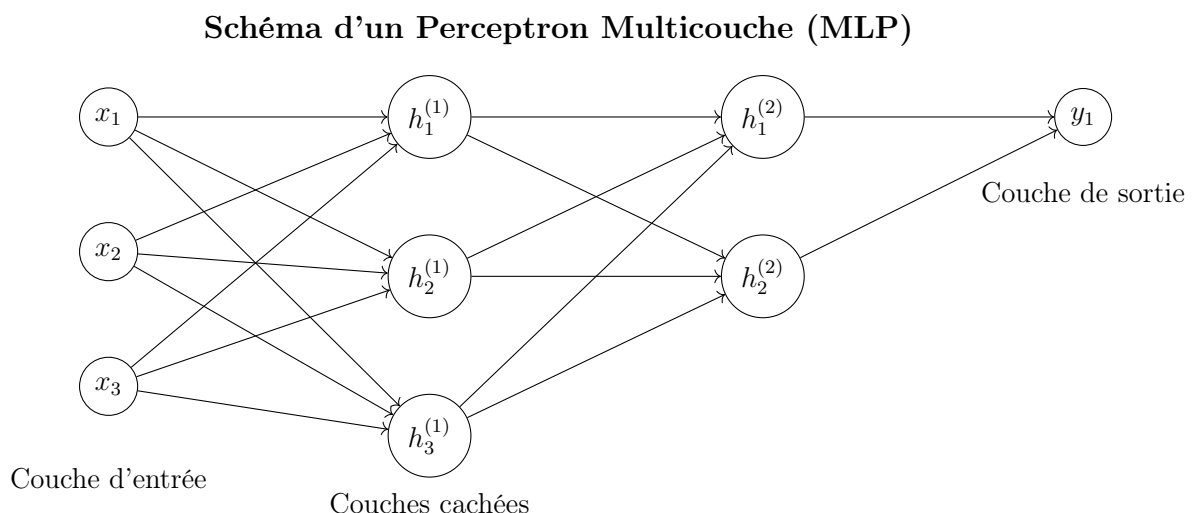


2.2.3 Perceptron Multicouche (MLP)

Le **Perceptron Multicouche (MLP)** est une version plus avancée du perceptron, composée d'une *couche d'entrée*, d'au moins une *couche cachée* et d'une *couche de sortie*. Chaque neurone est *pleinement connecté* à tous les neurones de la couche suivante. L'information circule uniquement de l'entrée vers la sortie, sans boucle : c'est ce qu'on appelle la *propagation avant*.

Chaque neurone calcule une *somme pondérée* de ses entrées, ajoute un *biais*, puis applique une *fonction d'activation* — généralement non linéaire, comme la sigmoïde ou ReLU — pour produire une sortie.

Grâce à ses couches cachées et à ses connexions, le MLP est capable de *modéliser des relations complexes* entre les données. Il est largement utilisé dans des tâches de **classification** et de **régression**, et constitue l'un des fondements des réseaux de neurones profonds.



Dans la suite de ce rapport, nous nous concentrerons principalement sur les réseaux de neurones **feedforward** (FFNN), en mettant en évidence leurs caractéristiques, leurs

architectures et leurs applications., l'objectif principal sera d'explorer en profondeur les réseaux feedforward et leur capacité d'approximation.

Concept	Neurone Artificiel	Perceptron
Définition	Unité de base d'un réseau de neurones. Il applique une somme pondérée et une fonction d'activation.	Modèle de neurone introduit par Frank Rosenblatt en 1958 pour la classification linéaire.
Formule	$y = f(\sum w_i x_i + b)$ avec une fonction d'activation f .	$y = \text{sgn}(\sum w_i x_i + b)$ avec une fonction seuil (sgn).
Fonction d'activation	ReLU, Sigmoid, Tanh, Softmax, etc.	Fonction de seuil $\text{sgn}(x)$ (binaire : 0 ou 1).
Apprentissage	Utilisé dans les MLPs et réseaux profonds.	Mise à jour des poids avec : $w_i^{(t+1)} = w_i^{(t)} + \eta(y_{\text{cible}} - y_{\text{prédit}})x_i$
Capacité	Approxime des fonctions complexes, gère des problèmes non linéaires .	Fonctionne uniquement pour les données linéairement séparables .
Utilisation	Composant fondamental de tous les réseaux de neurones.	Utilisé en classification binaire dans les années 50-60.

TABLE 2.6 – Comparaison entre un Neurone Artificiel et un Perceptron

2.3 Fonctions d'activation

Les **fonctions d'activation** permettent d'introduire de la non-linéarité dans les réseaux de neurones. Voici les plus utilisées :

2.3.1 Sigmoide

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (2.3)$$

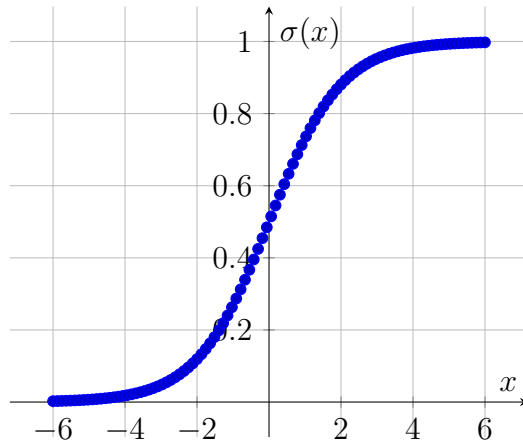


FIGURE 2.7 – Courbe de la fonction sigmoïde

2.3.2 ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x) \quad (2.4)$$

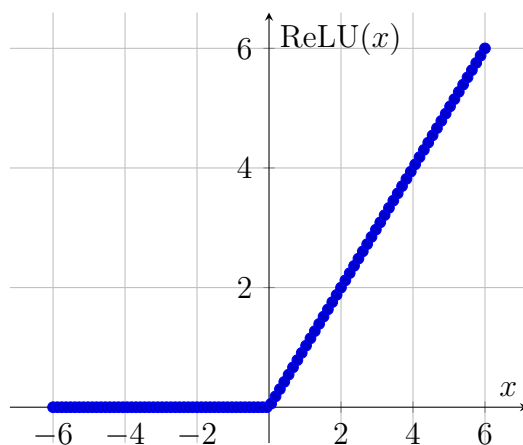
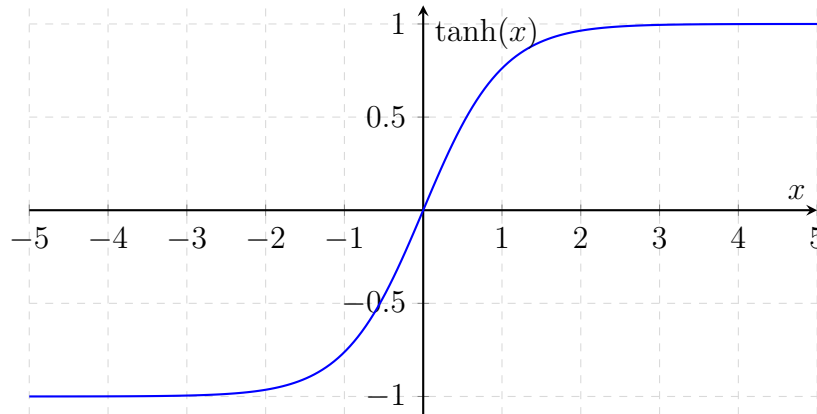


FIGURE 2.8 – Courbe de la fonction ReLU

2.3.3 Tanh (Tangente hyperbolique)

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (2.5)$$



2.3.4 Comparaison des fonctions d'activation

Fonction	Avantages	Inconvénients
Sigmoïde	Lisse, différentiable	Saturation pour valeurs extrêmes
ReLU	Simple, efficace	Problème de neurones morts
Tanh	Centré autour de 0	Saturation

TABLE 2.7 – Comparaison des fonctions d'activation

Les fonctions d'activation sont essentielles pour introduire de la non-linéarité dans les réseaux de neurones. Parmi les plus utilisées, on retrouve la sigmoïde, qui prend des valeurs dans l'intervalle $]0, 1[$. Cependant, elle souffre du phénomène du vanishing gradient lorsque x devient très grand ou très petit. La tangente hyperbolique, est centrée en zéro, avec une image dans $] - 1, 1[$, et est souvent utilisée dans les couches cachées. Toutefois, elle est également sujette au problème du gradient qui disparaît. Enfin, la fonction ReLU, a été largement adoptée dans les réseaux profonds en raison de sa simplicité et de son efficacité pour atténuer le phénomène du vanishing gradient pour les valeurs positives. Cependant, elle peut entraîner le phénomène de Dying ReLU pour des entrées négatives. Ainsi, le choix de la fonction d'activation dépend du contexte et de la tâche : la sigmoïde est privilégiée pour la classification binaire en sortie, la tangente hyperbolique pour les réseaux simples, et la ReLU pour des architectures plus complexes et profondes.

Théorèmes sur la Capacité d'Approximation des RNAs

3.1 Théorème d'Approximation Universelle

Un réseau de neurones à une couche cachée suffit pour approximer toute fonction continue sur un compact. Le **théorème d'approximation universelle** est l'un des résultats les plus fondamentaux et puissants en théorie des réseaux de neurones. Ce théorème stipule que les réseaux de neurones artificiels, en particulier ceux à une seule couche cachée, peuvent approximativement représenter toute fonction continue, sous certaines conditions. Ce résultat repose sur le fait que, avec suffisamment de neurones dans la couche cachée, un réseau de neurones feedforward avec une fonction d'activation non linéaire peut approcher toute fonction continue sur un domaine compact, avec une précision arbitraire.

Formellement, le théorème énonce que pour toute fonction continue $f : \mathbb{R}^n \rightarrow \mathbb{R}$, et pour tout $\epsilon > 0$, il existe un réseau de neurones à une seule couche cachée avec des poids appropriés et une fonction d'activation donnée (par exemple, la sigmoïde ou la tangente hyperbolique), tel que :

$$\left\| f(x) - \sum_{i=1}^N \alpha_i \sigma(w_i^T x + b_i) \right\| < \epsilon \quad \text{pour tout } x \in \mathbb{R}^n$$

où :

- σ est la fonction d'activation (telle que sigmoïde, tangente hyperbolique, etc.),
- w_i et b_i sont les poids et biais du réseau,
- α_i est un coefficient de pondération,
- N est le nombre de neurones dans la couche cachée.

Ce résultat est particulièrement remarquable car il montre que, bien que les réseaux de neurones soient des modèles paramétriques relativement simples, ils possèdent une capacité théorique extrêmement puissante pour approximer n'importe quelle fonction continue, à condition d'avoir suffisamment de neurones dans la couche cachée.

Dans la suite, nous allons explorer plus en détail ce théorème, ses preuves, ses limites pour la conception des réseaux de neurones, ainsi que les limitations et les développements ultérieurs nécessaires pour une mise en œuvre pratique efficace.

3.2 Théorème de Cybenko

3.2.1 Préliminaires et théorème

Définition 3.2.1: Hyperplan

Soit H un sous-espace de E , et $x \in E$, alors un ensemble donné par :

$$H = \{x \in E \mid \varphi(x) = \alpha\}, \quad \text{où } x \in \mathbb{R}$$

est un hyperplan.

Un hyperplan sépare strictement deux ensembles $A \subseteq E$ et $B \subseteq E$ si $\exists \varphi \in E^*$, $\alpha \in \mathbb{R}$, $\varepsilon > 0$ tels que :

$$\begin{cases} \varphi(x) < \alpha - \varepsilon & \forall x \in A \\ \varphi(x) > \alpha + \varepsilon & \forall x \in B \end{cases}$$

Définition 3.2.2: Espace des fonctions continues

Soit $I^n = [0, 1]^n$ le cube unité de dimension n . On note $C(I^n)$ l'ensemble des fonctions réelles continues définies sur I^n , c'est-à-dire :

$$C(I^n) = \{f : I^n \rightarrow \mathbb{R} \mid f \text{ continue}\}.$$

Définition 3.2.3: Norme uniforme ou norme infinie

Pour toute fonction $f \in C(I^n)$, on définit la **norme uniforme** (ou norme infinie) par :

$$\|f\|_\infty = \sup_{x \in I^n} |f(x)|.$$

C'est la valeur absolue maximale prise par la fonction sur tout le domaine I^n .

Définition 3.2.4: Espace des mesures de Borel

On note $M(I^n)$ l'espace des **mesures de Borel signées, régulières et de variation finie** sur le cube I^n . Les éléments de $M(I^n)$ sont des applications linéaires continues de $C(I^n)$ dans $\mathbb{R}$, au sens de la dualité classique en analyse fonctionnelle :

$$\mu : C(I^n) \rightarrow \mathbb{R}, \quad f \mapsto \int_{I^n} f(x) d\mu(x).$$

Définition 3.2.5: Fonction discriminante

Soit $I^n = [0, 1]^n$ le cube unité de dimension n , et soit $M(I^n)$ l'espace des mesures de Borel signées, régulières et de variation finie sur I^n .

On dit qu'une fonction $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ est **discriminante** si, pour toute mesure $\mu \in M(I^n)$,

$$\int_{I^n} \sigma(\langle y, x \rangle + \theta) d\mu(x) = 0 \quad \text{pour tout } y \in \mathbb{R}^n \text{ et } \theta \in \mathbb{R}$$

implique que $\mu = 0$.

Théorème 3.2.1: Hahn-Banach

Soient $A \subseteq E$, $B \subseteq E$ tels que $A \cap B = \emptyset$, supposons de plus que A , B sont convexes, A est fermé et B est compact. Alors :

$$\exists \varphi \in E^* \text{ tel que } \varphi = a \text{ qui sépare strictement } A \text{ et } B.$$

Corollaire 3.2.1

Soit $F \subset E$ un sous-espace vectoriel qui n'est pas dense dans E (i.e. $\overline{F} \neq E$). Alors :

$$\exists \varphi \in E^* \text{ tel que } \varphi \neq 0, \quad \varphi(x) = 0 \quad \forall x \in F$$

Théorème 3.2.2: Riesz-Markov

Soit X un espace topologique compact. Alors, pour toute forme linéaire continue $\varphi \in (C(X))^*$, il existe une mesure de Borel signée, régulière et de variation finie $\mu \in M(X)$ telle que :

$$\varphi(f) = \int_X f(x) d\mu(x), \quad \forall f \in C(X).$$

Autrement dit, l'espace dual $(C(X))^*$ est isomorphe à l'espace $M(X)$ des mesures de Borel signées, régulières et de variation finie sur X .

3.2.2 Théorème

Théorème 3.2.3: Cybenko, 1989

Soit $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ une fonction d'activation continue, non constante, bornée, telle que $\lim_{x \rightarrow -\infty} \sigma(x) = 0$ et $\lim_{x \rightarrow +\infty} \sigma(x) = 1$.

Alors, pour toute fonction continue $f \in C([0, 1]^n)$ et tout $\varepsilon > 0$, il existe un entier N , des poids $w_i \in \mathbb{R}^n$, des biais $b_i \in \mathbb{R}$, et des coefficients $\alpha_i \in \mathbb{R}$ tels que la fonction suivante :

$$F(x) = \sum_{i=1}^N \alpha_i \sigma(w_i \cdot x + b_i)$$

satisfasse :

$$\|f - F\|_{\infty} < \varepsilon,$$

c'est-à-dire que F approxime uniformément f sur $[0, 1]^n$

Application :

Le théorème de Cybenko établit la capacité des réseaux à une couche cachée à approximer toute fonction continue. Dans le cas des fonctions constantes définies par morceaux, une reproduction exacte est possible si l'on remplace la sigmoïde par la fonction de Heaviside. En combinant des Heavisides placées aux bons seuils, le réseau peut activer une constante précise sur chaque intervalle, reproduisant fidèlement la fonction cible. Voici un exemple illustrer pour les fonctions constantes par morceaux :

l'approximation de la fonction indicatrice d'un intervalle à l'aide de fonctions sigmoïdes, dans le cadre de l'approximation par réseaux de neurones :

Soit un intervalle $[a, b] \subset \mathbb{R}$ que l'on découpe en n sous-intervalles :

$$a = x_0 < x_1 < \dots < x_n = b$$

On définit la fonction $f(x)$ constante par morceaux sur cet intervalle comme suit :

$$f(x) = \begin{cases} c_1 & \text{si } x \in [x_0, x_1) \\ c_2 & \text{si } x \in [x_1, x_2) \\ \vdots & \\ c_{n-1} & \text{si } x \in [x_{n-2}, x_{n-1}) \\ c_n & \text{si } x \in [x_{n-1}, x_n] \end{cases}$$

avec $c_i \in \mathbb{R}$ des constantes.

Utilisation de la fonction de Heaviside

La fonction de Heaviside $H(x)$ est définie par :

$$H(x) = \begin{cases} 0 & \text{si } x < 0 \\ 1 & \text{si } x \geq 0 \end{cases}$$

On peut alors réécrire $f(x)$ comme suit :

$$f(x) = \sum_{i=1}^n c_i \cdot [H(x - x_{i-1}) - H(x - x_i)]$$

Explication des termes

- $H(x - x_{i-1}) - H(x - x_i)$ vaut :

- 1 si $x \in [x_{i-1}, x_i)$,
- 0 sinon.

- Le dernier terme $c_n \cdot H(x - x_{n-1})$ couvre l'intervalle $[x_{n-1}, x_n]$, où $f(x) = c_n$.

On conclure :

On peut donc exprimer toute fonction constante par morceaux sur un intervalle compact $[a, b]$, comme une combinaison linéaire de sorties de neurones avec activation Heaviside. Cela illustre concrètement le théorème de Cybenko : un réseau de neurones à une seule couche cachée avec suffisamment de neurones peut approximer toute fonction continue, et ici exactement reproduire une fonction par morceaux (qui est continue par morceaux).

3.2.3 Preuve du théorème de Cybenko

Objectif : Montrer que l'ensemble des MLP à une couche cachée, avec σ sigmoïdale, sont denses dans $C([0, 1]^n)$.

Méthode : Preuve par contraposée — nous allons supposer que l'ensemble des MLP à une couche cachée n'est pas dense, et montrer qu'alors la fonction d'activation σ ne peut pas être discriminante, ce qui contredit les hypothèses.

Étape 1 : Supposition de non-densité

Soit $\mathcal{H}_\sigma$ est l'ensemble des MLP à une couche cachée, c'est-à-dire :

$$\mathcal{H}_\sigma = \left\{ F(x) = \sum_{i=1}^N \alpha_i \sigma(w_i \cdot x + b_i) \mid N \in \mathbb{N}, \alpha_i \in \mathbb{R}, w_i \in \mathbb{R}^n, b_i \in \mathbb{R} \right\}.$$

On suppose que $\mathcal{H}_\sigma$ n'est **pas dense** dans $C(I^n)$ pour la norme uniforme :

$$\overline{\mathcal{H}_\sigma}^{\|\cdot\|_\infty} \neq C(I^n).$$

Étape 2 : Application du corollaire de Hahn-Banach

Par le corollaire du théorème de Hahn-Banach, s'il existe un sous-espace $F \subset C(I^n)$ qui n'est pas dense, alors il existe une forme linéaire continue non nulle $\varphi \in (C(I^n))^*$ telle que :

$$\varphi(f) = 0 \quad \forall f \in \mathcal{H}_\sigma.$$

Or, d'après le théorème de représentation de Riesz, toute forme linéaire continue sur $C(I^n)$ s'écrit comme :

$$\varphi(f) = \int_{I^n} f(x) d\mu(x),$$

où $\mu \in M(I^n)$ est une mesure signée de Borel, régulière et de variation finie.

Ainsi, il existe $\mu \in M(I^n)$, $\mu \neq 0$, telle que :

$$\int_{I^n} \sum_{i=1}^N \alpha_i \sigma(w_i \cdot x + b_i) d\mu(x) = 0 \quad \forall F \in \mathcal{H}_\sigma.$$

Par linéarité, cela implique que pour toute paire $(w, b) \in \mathbb{R}^n \times \mathbb{R}$,

$$\int_{I^n} \sigma(w \cdot x + b) d\mu(x) = 0.$$

Étape 3 : Contradiction avec le caractère discriminant de σ

D'après la définition, si la fonction σ est **discriminante**, alors :

$$\left(\forall (w, b) \in \mathbb{R}^n \times \mathbb{R}, \quad \int_{I^n} \sigma(w \cdot x + b) d\mu(x) = 0 \right) \Rightarrow \mu = 0.$$

Mais nous avons construit une mesure $\mu \neq 0$ qui satisfait :

$$\int_{I^n} \sigma(w \cdot x + b) d\mu(x) = 0 \quad \forall (w, b).$$

Cela contredit donc le caractère discriminant de σ .

Conclusion

L'hypothèse selon laquelle $\mathcal{H}_\sigma$ n'est pas dense dans $C(I^n)$ conduit à une contradiction. Par contraposée, on conclut que :

$$\overline{\mathcal{H}_\sigma}^{\|\cdot\|_\infty} = C(I^n),$$

c'est-à-dire que l'ensemble des MLP à une couche cachée est dense dans l'espace des fonctions continues.

3.2.4 Limites du théorème de Cybenko

Le théorème de Cybenko constitue un résultat fondamental dans la théorie de l'approximation par réseaux de neurones. Il établit que les réseaux de neurones à une seule couche cachée, munis d'une fonction d'activation sigmoïdale, sont capables d'approximer uniformément toute fonction continue sur le cube unité $I^n = [0, 1]^n$. Toutefois, ce théorème, bien que puissant, présente plusieurs limitations importantes que nous présentons ci-dessous.

- **Approximation restreinte aux fonctions continues** : Le théorème garantit uniquement l'approximation des fonctions dans l'espace $C(I^n)$, c'est-à-dire les fonctions réelles continues sur le cube unité. Il ne traite pas du cas des fonctions non continues ou mesurables, ni de l'approximation dans d'autres espaces fonctionnels tels que $L^p(I^n)$.
- **Résultat non constructif** : Le théorème affirme l'existence d'un réseau approximant, mais il ne fournit aucune méthode explicite pour déterminer les paramètres (poids, biais, coefficients). Il ne donne pas non plus de borne sur le nombre de neurones nécessaires pour atteindre une erreur ε donnée.
- **Absence d'estimation de complexité** : Aucune information n'est donnée quant à la complexité du réseau (nombre de neurones, taille des coefficients, etc.) en fonction de la régularité de la fonction cible ou du niveau de précision souhaité.
- **Limitation aux architectures peu profondes** : Le cadre du théorème est celui des réseaux à une seule couche cachée. Or, en pratique, les réseaux profonds (comportant plusieurs couches cachées) montrent souvent de meilleures performances, tant en termes de capacité d'approximation que d'efficacité numérique.
- **Hypothèse restrictive sur la fonction d'activation** : Le théorème ne concerne que les fonctions d'activation sigmoïdales (bornées, continues, strictement croissantes, asymptotiques en $\pm\infty$). Des fonctions d'activation non bornées telles que ReLU ou ses variantes ne sont pas couvertes, bien qu'elles soient largement utilisées en pratique.

- **Portée purement théorique :** Le résultat est asymptotique et théorique. Il ne garantit aucunement que des méthodes d'apprentissage usuelles (descente de gradient, rétropropagation, etc.) permettent d'atteindre l'approximation souhaitée. En effet, les réseaux de neurones sont souvent sujets à des problèmes d'optimisation tels que l'existence de minima locaux ou de plateaux.
- **Cas particulier des fonctions scalaires :** Le théorème traite uniquement du cas des fonctions à valeurs scalaires. Il ne s'étend pas directement au cas des fonctions vectorielles, où la sortie est de dimension supérieure à un. Dans ce cas, il faut considérer des approximations composantes par composantes.

Ainsi, bien que le théorème de Cybenko établisse une première base théorique solide pour l'approximation universelle par réseaux de neurones, son application pratique reste limitée et nécessite d'être complétée par d'autres résultats plus généraux, notamment ceux de Hornik (1991), Barron (1993), ou encore les résultats sur les réseaux profonds récents. ■

3.3 Théorème de Hornik : Extension aux fonctions continues

Le théorème de Cybenko a montré que les réseaux de neurones à une couche cachée, munis d'une fonction d'activation sigmoïdale, peuvent approximer uniformément toute fonction continue définie sur cube unité. Toutefois, ce résultat repose sur des hypothèses spécifiques, notamment le caractère sigmoïdal de la fonction d'activation.

Dans ce contexte, le travail de Kurt Hornik (1991) constitue une généralisation importante. Il démontre que la capacité d'approximation universelle ne dépend pas exclusivement du caractère sigmoïdal, mais d'une propriété plus large des fonctions d'activation : il suffit qu'elles soient non polynomiales pour que le réseau soit capable d'approximer des fonctions arbitraires.

Ainsi, le théorème de Hornik étend la portée du théorème de Cybenko et établit que, sous des conditions beaucoup plus générales, les réseaux de neurones multicouches sont toujours des estimateurs universels. Il fournit également une justification théorique à l'utilisation d'activations modernes telles que la tangente hyperbolique, ou même ReLU, dans certains contextes.

Dans les sections qui suivent, nous détaillerons les énoncés et implications du théorème de Hornik, ainsi que sa portée en approximation.

3.3.1 Énoncé rigoureux du théorème

Théorème 3.3.1: Hornik

Soit $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ une fonction d'activation continue, bornée et non constante. Alors, pour tout compact $K \subset \mathbb{R}^n$, l'ensemble des fonctions

$$\mathcal{F} = \left\{ F(x) = \sum_{i=1}^N \alpha_i \sigma(w_i^T x + b_i) \mid N \in \mathbb{N}, \alpha_i \in \mathbb{R}, w_i \in \mathbb{R}^n, b_i \in \mathbb{R} \right\}$$

est dense dans $\mathcal{C}(K)$ au sens de la norme $\|\cdot\|_\infty$. Autrement dit, pour toute fonction

$f \in \mathcal{C}(K)$ et tout $\varepsilon > 0$, il existe une fonction $F \in \mathcal{F}$ telle que

$$\|f - F\|_\infty < \varepsilon.$$

3.3.2 Preuve du théorème de Hornik

Étape 1 : Hypothèse de non-densité. Supposons par l'absurde que l'ensemble $\mathcal{F}$ n'est pas dense dans $C(K)$. Alors il existe une fonction $f \in C(K)$ et un $\varepsilon > 0$ tels que :

$$\forall F \in \mathcal{F}, \quad \|f - F\|_\infty \geq \varepsilon.$$

Étape 2 : Théorème de Hahn-Banach et représentation de Riesz. Par le théorème de Hahn-Banach (forme du théorème de séparation), il existe une forme linéaire continue $L \in C(K)^*$ telle que :

$$L(F) = 0 \quad \text{pour tout } F \in \mathcal{F}, \quad L(f) \neq 0.$$

Par le théorème de représentation de Riesz, il existe une mesure signée de Radon $\mu \neq 0$ sur K telle que :

$$L(g) = \int_K g(x) d\mu(x), \quad \text{pour tout } g \in C(K).$$

En particulier, pour tout $w \in \mathbb{R}^n$ et $b \in \mathbb{R}$, on a :

$$\int_K \sigma(w^T x + b) d\mu(x) = 0.$$

Étape 3 : Contradiction – Propriété de séparation. Nous invoquons le résultat suivant :

Proposition 3.3.1: Séparation par les fonctions sigmoïdes, Hornik

Si $\sigma \in C(\mathbb{R})$ est bornée et non constante, alors l'ensemble $\{x \mapsto \sigma(w^T x + b)\}$ sépare les mesures. Autrement dit,

$$\int_K \sigma(w^T x + b) d\mu(x) = 0 \quad \forall w, b \in \mathbb{R}^n \Rightarrow \mu = 0.$$

Cette propriété découle du fait que les fonctions $\sigma(w^T x + b)$ engendrent une algèbre suffisamment riche pour approcher toute fonction continue sur K , si σ n'est pas polynomiale.

Cela contredit notre hypothèse que $\mu \neq 0$. Par conséquent, l'ensemble $\mathcal{F}$ est dense dans $C(K)$. □

Remarque

Ce théorème généralise le résultat de Cybenko (1989), qui traitait uniquement du cas des fonctions sigmoïdes. Hornik montre que la capacité d'approximation universelle ne dépend pas de la forme exacte de σ , mais du fait qu'elle soit non constante (et dans d'autres versions, non polynomiale).

3.4 Comparaison des résultats et implications pratiques

Après avoir présenté les théorèmes de Cybenko et de Hornik, il est pertinent de comparer leurs hypothèses, leurs portées respectives ainsi que leurs implications concrètes dans la pratique des réseaux de neurones.

Élément	Théorème de Cybenko (1989)	Théorème de Hornik (1991)
Type de fonction σ	Continue, sigmoïde (valeurs dans $[0, 1]$)	Continue, bornée, non constante
Domaine	Compact $[0, 1]^n \subset \mathbb{R}^n$	Compact $K \subset \mathbb{R}^n$
Résultat	Approximation uniforme dans $C([0, 1]^n)$	Approximation uniforme dans $C(K)$
Méthodologie	Théorème de Hahn-Banach + dualité fonctionnelle	Argument fonctionnel + propriété d'approximation algébrique
Limitation	Applicable uniquement aux fonctions sigmoïdes	S'applique à un large éventail de fonctions

TABLE 3.1 – Comparaison entre les théorèmes de Cybenko et de Hornik

3.4.1 Conséquences mathématiques

Le théorème de Cybenko établit la capacité d'approximation universelle pour les réseaux de neurones à une seule couche cachée utilisant une fonction d'activation sigmoïde. Le théorème de Hornik généralise ce résultat en montrant que toute fonction continue, bornée et non constante suffit pour garantir l'universalité de l'approximation.

3.4.2 Implications pratiques

- Le choix de la fonction d'activation peut être beaucoup plus libre qu'on ne le pensait à l'origine : `ReLU`, `tanh`, etc., sont tous admissibles sous certaines conditions.
- Les fonctions d'activation polynomiales sont à proscrire car elles ne permettent pas l'universalité de l'approximation.
- Ces théorèmes justifient théoriquement l'usage des MLPs à une seule couche cachée, mais ne garantissent ni l'efficacité de l'apprentissage ni la généralisation.

3.4.3 Portée et limites en pratique

Ces résultats théoriques ne tiennent pas compte de :

- l'efficacité de l'apprentissage (convergence de l'optimisation),
- la quantité de neurones nécessaires,
- les capacités de généralisation sur données réelles,
- les avantages potentiels de la profondeur.

Bien que la profondeur ne soit pas nécessaire à l'universalité de l'approximation, des travaux récents ont montré qu'elle permet une meilleure efficacité dans certains cas.

3.4.4 Conclusion du chapitre

Dans ce chapitre, nous avons exploré la capacité des réseaux de neurones à approximer des fonctions continues sur des compacts de $\mathbb{R}^n$. À travers les théorèmes fondamentaux de Cybenko et de Hornik, nous avons démontré que les réseaux à une seule couche cachée dotés de fonctions d'activation appropriées sont des approximateurs universels.

Le théorème de Cybenko a posé les premières bases de cette propriété en se limitant aux fonctions sigmoïdes, tandis que le théorème de Hornik a étendu cette universalité à un cadre beaucoup plus large, incluant toute fonction continue, bornée et non constante. Ces résultats assurent qu'en théorie, un réseau de neurones peut approximer toute fonction continue avec une précision arbitraire, pourvu qu'il soit suffisamment grand.

Cependant, cette puissance théorique ne garantit ni l'efficacité de l'entraînement, ni la généralisation sur des données réelles. En effet, savoir qu'une fonction peut être approchée est une chose ; trouver effectivement les paramètres du réseau qui réalisent cette approximation en est une autre. C'est tout l'enjeu de l'apprentissage.

Dans le chapitre suivant, nous allons aborder en détail les méthodes d'apprentissage des réseaux de neurones, en commençant par la définition des fonctions de perte et la méthode de rétropropagation du gradient, éléments essentiels à l'entraînement effectif des modèles neuronaux.

Chapitre 4

Méthodes d'Apprentissage des Réseaux de Neurons

Introduction

L'apprentissage est un processus fondamental dans le domaine de l'intelligence artificielle et plus particulièrement dans les réseaux de neurones artificiels. Il s'agit du mécanisme par lequel un réseau ajuste automatiquement ses paramètres internes à partir de données afin d'accomplir une tâche précise. Le succès des réseaux de neurones dans des domaines variés comme la reconnaissance d'image, la traduction automatique ou encore la prédiction financière repose en grande partie sur l'efficacité des méthodes d'apprentissage utilisées. Ce chapitre présente les principales approches d'apprentissage, leurs objectifs et les mécanismes sous-jacents qui permettent à un réseau d'apprendre à partir des données.

4.1 Notions fondamentales de l'apprentissage

L'apprentissage automatique (*Machine Learning*) est un sous-domaine de l'intelligence artificielle qui consiste à concevoir des systèmes capables d'améliorer leurs performances à partir d'exemples ou de données expérimentales, sans être explicitement programmés pour chaque tâche.

4.1.1 Objectifs de l'apprentissage

Les principaux objectifs de l'apprentissage sont les suivants :

- **Minimisation d'une fonction de coût** : L'apprentissage consiste à ajuster les paramètres du modèle pour minimiser une mesure de l'erreur (fonction de coût) entre les prédictions et les valeurs attendues.
- **Généralisation** : Un bon modèle n'est pas seulement performant sur les données d'entraînement, mais aussi sur des données jamais vues. C'est cette capacité à bien se comporter sur des données nouvelles qui constitue la généralisation.
- **Performance** : Selon la tâche (régression, classification, etc.), la performance peut être mesurée à l'aide de métriques telles que l'exactitude (accuracy), la précision, le rappel, l'erreur quadratique moyenne, etc.

4.1.2 Types d'apprentissage

Il existe plusieurs paradigmes d'apprentissage automatique :

Apprentissage supervisé

L'apprentissage supervisé s'effectue à partir d'une base de données particulière, appelée **base d'apprentissage**, constituée d'un ensemble fini de couples de la forme (x_i, y_i) , où :

- x_i représente une *entrée* (ou vecteur de caractéristiques),
- y_i désigne la *sortie désirée* ou *étiquette* associée à cette entrée.

Les sorties désirées définissent un **étiquetage** qui permet de subdiviser les entrées en **classes prédéfinies**. Ainsi, les exemples fournis au modèle sont connus, et les classes auxquelles ils appartiennent sont explicitement spécifiées. Le système a alors pour objectif d'apprendre à **associer correctement chaque entrée à sa classe**, selon un modèle de classification donné.

Ce processus d'apprentissage vise non seulement à reproduire correctement les sorties attendues pour les exemples d'apprentissage, mais aussi à **généraliser** cette connaissance afin de pouvoir **prédire la classe de nouvelles données** jamais vues auparavant. Pour cela, le modèle s'appuie sur des techniques issues de la **régression statistique** et de l'**optimisation**.

Apprentissage non supervisé

Comme son nom l'indique, l'apprentissage **non supervisé** repose sur un autre type de base de données, appelée **base de test**, dans laquelle **aucune sortie désirée n'est disponible** : seuls les vecteurs d'entrée sont fournis.

Ce type d'apprentissage est principalement utilisé dans des tâches spécifiques telles que le **clustering** (regroupement de données), ou encore la **répartition automatique** des données. L'objectif est alors de regrouper des exemples potentiellement hétérogènes en collections ou groupes homogènes, où les éléments partagent des caractéristiques communes.

Dans ce cadre, **aucune intervention extérieure n'est fournie au système** concernant les structures recherchées ou les groupes attendus. C'est donc à la machine de **découvrir seule des régularités ou des motifs cachés** au sein des données.

La similarité entre les données est généralement mesurée à l'aide d'une **fonction de distance** définie sur l'espace des entrées, permettant d'évaluer à quel point deux exemples sont proches ou éloignés l'un de l'autre.

Apprentissage par renforcement

L'apprentissage **par renforcement** repose sur trois éléments fondamentaux : **l'agent**, **l'environnement** et le **système de récompense**.

Dans ce cadre, l'agent interagit avec l'environnement : à chaque instant, il observe un **état** de l'environnement, en déduit une **action** à effectuer, puis reçoit en retour une **récompense**, qui évalue positivement ou négativement la qualité de cette action.

L'objectif de l'agent est de **maximiser la récompense cumulée** au cours du temps. Cela implique qu'il doive progressivement **apprendre à distinguer les bonnes des mauvaises actions**, et ainsi favoriser celles qui mènent à des résultats favorables. Ce processus d'apprentissage repose sur la notion d'exploration (essayer différentes actions) et d'exploitation (choisir les meilleures actions déjà connues).

Cette approche vise à reproduire les **mécanismes naturels d'apprentissage** observés chez les êtres vivants. Ce qui la rend particulièrement puissante est qu'elle **ne nécessite pas de base de données étiquetée** comme dans les apprentissages supervisé ou non supervisé : le système apprend uniquement par essais-erreurs et par interaction avec l'environnement.

Autres types d'apprentissage

- **Apprentissage semi-supervisé** : Combinaison de données étiquetées et non étiquetées.
- **Apprentissage par transfert** : Réutilisation de connaissances acquises dans une tâche source pour améliorer l'apprentissage d'une tâche cible.

Dans la suite du chapitre, nous allons nous concentrer sur l'apprentissage supervisé. Ce paradigme est le plus couramment utilisé dans les réseaux de neurones, notamment pour des tâches de classification (reconnaissance d'image, NLP) ou de régression. Nous présenterons en détail les mécanismes qui le sous-tendent : fonction de coût, ajustement des paramètres, et algorithmes d'optimisation comme la descente de gradient.

4.2 Optimisation et apprentissage supervisé d'un neurone formel

4.2.1 Formulation du problème

L'objectif de l'apprentissage supervisé est d'estimer une fonction inconnue f à partir d'un ensemble d'exemples d'apprentissage $\{(X^{(h)}, d(h))\}_{h=1}^{\ell}$, où chaque $X^{(h)} \in \mathbb{R}^d$ est une entrée (ou vecteur de caractéristiques) et $d(h)$ la sortie désirée associée. Un réseau de neurones, représenté par une fonction paramétrée $g(\cdot, \omega)$, a pour but d'approcher f aussi fidèlement que possible. L'apprentissage consiste alors à ajuster les paramètres ω (poids et biais du réseau) de manière à minimiser une fonction de coût globale.

$$C(\omega) = \frac{1}{\ell} \sum_{h=1}^{\ell} \mathcal{L}(d(h), g(X^{(h)}, \omega)),$$

4.2.2 Problème de régression

Le problème de régression consiste à approximer une fonction inconnue $f : \mathbb{R}^n \rightarrow \mathbb{R}$ à partir d'un ensemble d'exemples d'apprentissage $\{(X^{(h)}, d(h))\}_{h=1}^{\ell}$, où chaque $X^{(h)} \in \mathbb{R}^n$ est un vecteur d'entrée, et $d(h) \in \mathbb{R}$ est la sortie réelle associée.

Le but est de déterminer un ensemble de paramètres ω pour un modèle $g(X, \omega)$, tel que :

$$g(X^{(h)}, \omega) \approx d(h), \quad \text{pour tout } h = 1, \dots, \ell.$$

Autrement dit, on cherche à minimiser l'erreur entre la sortie réelle du modèle et la sortie désirée sur l'ensemble des exemples, en utilisant typiquement l'erreur quadratique moyenne :

$$C(\omega) = \frac{1}{\ell} \sum_{h=1}^{\ell} \frac{1}{2} (d(h) - g(X^{(h)}, \omega))^2.$$

Cette tâche s'inscrit dans un cadre d'apprentissage supervisé, où la connaissance explicite des sorties désirées $d(h)$ permet d'orienter l'ajustement des paramètres du modèle.

Exemple de régression par régression linéaire

Dans cette partie, nous utilisons le modèle de régression linéaire multiple (Linear Regression) pour effectuer une tâche de prédiction sur un jeu de données contenant des informations sur des logements. Ce jeu de données contient plusieurs caractéristiques, notamment la superficie du logement, le nombre de chambres et son âge, afin de prédire son prix.

Le code utilisé pour charger les données, traiter les valeurs manquantes, entraîner le modèle de régression et effectuer une prédiction du prix d'un logement est présenté ci-dessous.

```

1 import pandas as pd
2 import numpy as np
3 from sklearn import linear_model
4 import matplotlib.pyplot as plt
5
6 df = pd.read_csv('homeprices_1.csv')
7 print(df)
8
9 print(df.isnull().sum())
10
11 med = np.floor(df.bedrooms.mean())
12 print(med)
13
14 df.bedrooms = df.bedrooms.fillna(med)
15 print(df)
16
17 reg = linear_model.LinearRegression()
18
19 reg.fit(df[['area', 'bedrooms', 'age']], df.price)
20
21 print(reg.coef_)
22 print(reg.intercept_)
23
24 print(reg.predict([[3600, 3, 30]]))

```

Listing 4.1 – Prédiction des prix immobiliers avec Linear Regression

Résultats

Après l'exécution du programme, nous obtenons les résultats suivants :

	area	bedrooms	age	price
2	0	2600	3.0	20 550000
3	1	3000	4.0	15 565000

```

4 2      3200      NaN    18    610000
5 3      3600      3.0    30    595000
6 4      4000      5.0     8    760000
7 5      4100      6.0     8    810000
8
9 area          0
10 bedrooms     1
11 age          0
12 price        0
13
14 4.0
15
16 [112.06244194 23388.88007794 -3231.71790863]
17
18 221323.00186540408
19
20 [597962.89583192]

```

Listing 4.2 – Résultats obtenus lors de l'exécution

Après l'exécution du programme, le modèle de régression linéaire obtient les coefficients suivants :

[112.0624 , 23388.8801 , 3231.7179]

ainsi qu'une constante de :

221323.0019

Pour une maison ayant une superficie de 3600 pieds carrés, 3 chambres et un âge de 30 ans, le modèle prédit un prix de :

597962.90

Ainsi, le modèle estime le prix de cette maison à environ 597,963.

4.2.3 Problème de classification

Le problème de classification consiste à associer à chaque vecteur d'entrée $X^{(h)} \in \mathbb{R}^n$ une étiquette de classe parmi un ensemble fini de K classes. Chaque exemple d'apprentissage est donné sous la forme $(X^{(h)}, d^{(h)})$, où $d^{(h)} = (d_1^{(h)}, \dots, d_K^{(h)}) \in \{0, 1\}$ est un vecteur codé en one-hot représentant la classe correcte.

L'objectif est d'ajuster les paramètres ω d'un modèle $g(X, \omega)$, produisant une sortie $g(X^{(h)}, \omega) = (g_1(X^{(h)}, \omega), \dots, g_K(X^{(h)}, \omega))$, telle que :

$$g_k(X^{(h)}, \omega) \approx d_k^{(h)}, \quad \text{pour tout } h = 1, \dots, \ell \text{ et } k = 1, \dots, K.$$

La sortie du modèle est généralement interprétée comme une distribution de probabilité sur les classes, obtenue via une fonction softmax en sortie du réseau.

Le critère d'apprentissage utilisé est souvent l'entropie croisée, qui mesure la dissimilarité entre la distribution prédite et la distribution cible :

$$C(\omega) = -\frac{1}{\ell} \sum_{h=1}^{\ell} \sum_{k=1}^K d_k^{(h)} \log(g_k(X^{(h)}, \omega)).$$

Ce cadre suppose également un apprentissage supervisé, dans lequel chaque exemple d'apprentissage est étiqueté avec sa classe correcte.

Exemple de Classification par régression logistique

Dans cette partie, nous utilisons le modèle de régression logistique (*Logistic Regression*) pour effectuer une tâche de classification sur le jeu de données *Digits* de Scikit-learn. Ce jeu de données contient des images de chiffres manuscrits allant de 0 à 9.

Le code utilisé pour charger les données, entraîner le modèle et évaluer ses performances est présenté ci-dessous.

```
1 import matplotlib.pyplot as plt
2 from sklearn.datasets import load_digits
3 from sklearn.model_selection import train_test_split
4 from sklearn.linear_model import LogisticRegression
5 from sklearn.metrics import confusion_matrix, accuracy_score
6
7 # Chargement des données
8 digits = load_digits()
9
10 # Visualisation de quelques chiffres
11 for i in range(2):
12     plt.gray()
13     plt.matshow(digits.images[i])
14     print("Classe :", digits.target[i])
15
16 # Séparation des données
17 X_train, X_test, y_train, y_test = train_test_split(
18     digits.data,
19     digits.target,
20     test_size=0.2,
21     random_state=42,
22     stratify=digits.target
23 )
24
25 # Création et entraînement du modèle
26 model = LogisticRegression(max_iter=1000)
27 model.fit(X_train, y_train)
28
29 # Prédiction
30 y_predicted = model.predict(X_test)
31
32 # Évaluation
33 accuracy = accuracy_score(y_test, y_predicted)
34 print("Accuracy :", accuracy)
35
36 # Matrice de confusion
37 cm = confusion_matrix(y_test, y_predicted)
38 print("Matrice de confusion :")
39 print(cm)
```

Listing 4.3 – Classification des chiffres avec Logistic Regression

Résultats

Après l'exécution du programme, nous obtenons les résultats suivants :

```

1 Classe : 0
2 Classe : 1
3
4 Accuracy : 0.9583333333333334
5
6 Matrice de confusion :
7 [[35  0  1  0  0  0  0  0  0  0]
8  [ 0 32  0  1  0  0  0  0  3  0]
9  [ 0  0 35  0  0  0  0  0  0  0]
10 [ 0  0  0 37  0  0  0  0  0  0]
11 [ 0  0  0  0 36  0  0  0  0  0]
12 [ 0  0  0  1  0 36  0  0  0  0]
13 [ 0  0  0  0  0  0 35  0  1  0]
14 [ 0  0  0  0  1  0  0 35  0  0]
15 [ 0  4  0  0  0  1  0  0 30  0]
16 [ 0  0  0  0  1  0  0  0  1 34]]

```

Listing 4.4 – Résultats obtenus lors de l'exécution

La précision obtenue par le modèle est de :

$$\text{Accuracy} = 0.9583 \approx 95.83\%$$

Cela signifie que le modèle a correctement classifié environ 95.83 % des images de l'ensemble de test.

La matrice de confusion permet d'analyser plus précisément les prédictions du modèle. Les éléments situés sur la diagonale principale correspondent aux classifications correctes, tandis que les éléments situés en dehors de la diagonale représentent les erreurs de classification.

On remarque que la majorité des valeurs sont concentrées sur la diagonale, ce qui indique que le modèle possède de bonnes performances pour reconnaître les différents chiffres. Certaines confusions sont cependant présentes, notamment entre les classes 1 et 8, ainsi qu'entre les classes 8 et 1.

Origine de l'Entropie Croisée via le Maximum de Vraisemblance

Dans le cadre de l'apprentissage supervisé, considérons :

— Un jeu de données $\{(X^{(h)}, d^{(h)})\}_{h=1}^{\ell}$ où :

$$X^{(h)} \in \mathbb{R}^n \text{ (vecteur d'entrée)}$$

$$d^{(h)} = (d_1^{(h)}, \dots, d_K^{(h)}) \in \{0, 1\} \text{ (vecteur one-hot)}$$

— Un modèle $g(X, \omega)$ produisant une distribution de probabilité :

$$g(X^{(h)}, \omega) = (g_1(X^{(h)}, \omega), \dots, g_K(X^{(h)}, \omega))$$

avec $\sum_{k=1}^K g_k = 1$ via fonction softmax.

Lien avec la Vraisemblance

La vraisemblance des données s'écrit :

$$\mathcal{L}(\omega) = \prod_{h=1}^{\ell} P(d^{(h)} | X^{(h)}; \omega) = \prod_{h=1}^{\ell} \prod_{k=1}^K g_k(X^{(h)}, \omega)^{d_k^{(h)}}$$

En passant à la log-vraisemblance :

$$\log \mathcal{L}(\omega) = \sum_{h=1}^{\ell} \sum_{k=1}^K d_k^{(h)} \log g_k(X^{(h)}, \omega)$$

Formulation de l'Entropie Croisée

On définit la fonction de coût :

$$C(\omega) = -\frac{1}{\ell} \log \mathcal{L}(\omega) = -\frac{1}{\ell} \sum_{h=1}^{\ell} \sum_{k=1}^K d_k^{(h)} \log g_k(X^{(h)}, \omega)$$

4.2.4 Fonctions de coût classiques

Dans l'apprentissage supervisé, le but est d'ajuster les poids ω d'un réseau de neurones de manière à minimiser l'erreur entre la sortie désirée $d(h)$ et la sortie réelle du modèle $g(X^{(h)}, \omega)$, pour chaque exemple d'apprentissage $X^{(h)}$, $h = 1, \dots, \ell$.

Le choix de la fonction de coût (ou fonction de perte) dépend du type de tâche : régression ou classification.

Tâche	Fonction de coût	Avantages / Remarques
Régression	$C(\omega) = \frac{1}{\ell} \sum_{h=1}^{\ell} \frac{1}{2} (d(h) - g(X^{(h)}, \omega))^2$	Simple, différentiable, sensible aux outliers
Classification	$C(\omega) = -\frac{1}{\ell} \sum_{h=1}^{\ell} \sum_{k=1}^K d_k^{(h)} \log g_k(X^{(h)}, \omega)$	Appropriée pour les probabilités, efficace avec softmax

TABLE 4.1 – Comparaison des fonctions de coût selon le type de tâche

4.2.5 Formulation et optimisation

Considérons un perceptron qui prend la forme suivante :

Les entrées. Les entrées sont des réels regroupés sous la forme d'un vecteur :

$$x = (x_0, x_1, \dots, x_n)^\top \in \mathbb{R}^{n+1}$$

où $x_0 = 1$ est une entrée fictive introduite pour modéliser le biais ω_0 .

Les poids. Les poids sont également représentés par un vecteur :

$$\omega = (\omega_0, \omega_1, \dots, \omega_n)^\top \in \mathbb{R}^{n+1}$$

Le poids ω_0 joue le rôle de biais

La fonction d'entrée totale. Il s'agit d'une somme pondérée des entrées :

$$u = \omega^\top x = \sum_{i=0}^n \omega_i x_i = \omega_0 + \sum_{i=1}^n \omega_i x_i = \langle \tilde{\mathbf{x}}, \boldsymbol{\omega} \rangle,$$

Où l'on considère :

$x_0 = 1$ pour incorporer le biais dans le produit scalaire.

$\tilde{\mathbf{x}} = (1, x_1, \dots, x_n)$ est le vecteur d'entrée étendu.

La sortie du perceptron est alors : $S = f(u)$, où f est une fonction d'activation (ex. $\tanh(\cdot)$, $\text{sign}(\cdot)$, etc.).

Cadre de l'apprentissage supervisé

Dans ce contexte, on suppose que l'on dispose d'une base d'apprentissage

$$\mathcal{B} = \{(\mathbf{X}^{(h)}, d^{(h)})\}_{1 \leq h \leq \ell},$$

où :

- ℓ est la taille de la base d'apprentissage ;
- Pour chaque $h \in \{1, \dots, \ell\}$, $\mathbf{X}^{(h)} = (x_1^{(h)}, \dots, x_n^{(h)}) \in \mathbb{R}^n$ est un exemple d'entrée, et $d^{(h)} \in \mathbb{R}$ est la sortie désirée associée.

Objectif d'apprentissage : critère des moindres carrés

L'objectif est d'ajuster les poids $\boldsymbol{\omega}$ pour que la sortie du perceptron approche au mieux les sorties désirées. On utilise généralement le critère des moindres carrés (Least Mean Squares, LMS), qui consiste à résoudre le problème suivante :

$$(P) \quad \min_{\boldsymbol{\omega} \in \mathbb{R}^{n+1}} C(\boldsymbol{\omega}) = \frac{1}{2\ell} \sum_{h=1}^{\ell} (d^{(h)} - g(\mathbf{X}^{(h)}, \boldsymbol{\omega}))^2.$$

où $g(\mathbf{X}^{(h)}, \boldsymbol{\omega})$ est la sortie réelle du perceptron.

4.2.6 Remarques

1. On distingue généralement deux types d'erreurs :
— **Erreur locale** pour un exemple h :

$$e^{(h)}(\omega) = \frac{1}{2} (d^{(h)} - g(\mathbf{X}^{(h)}, \omega))^2;$$

- **Erreur totale** sur la base :

$$E(\omega) = \sum_{h=1}^{\ell} e^{(h)}(\omega) = \sum_{h=1}^{\ell} \frac{1}{2} (d^{(h)} - g(\mathbf{X}^{(h)}, \omega))^2;$$

- **Erreur moyenne** :

$$C(\omega) = \frac{1}{\ell} E(\omega).$$

2. On a $E(\omega) = 0$ si et seulement si $e^{(h)}(\omega) = 0$ pour tout h , ce qui signifie que le modèle correspond parfaitement aux données.
3. L'introduction du facteur $\frac{1}{\ell}$ dans la fonction coût $C(\omega)$ permet de faciliter l'extension au cadre probabiliste, notamment pour justifier la convergence vers l'espérance théorique en invoquant le théorème central limite. Dans un cadre strictement déterministe, ce facteur peut être omis.

4.2.7 Algorithmes d'optimisation

À ce stade, on a bien vu que le principe général de l'apprentissage supervisé pour un neurone formel consiste à trouver un élément neuronal optimal qui approche le mieux notre base de données. Cela revient, comme mentionné précédemment, à la résolution du problème d'optimisation suivant :

$$\min_{\omega \in \mathbb{R}^{n+1}} E(\omega) = \frac{1}{2} \sum_{h=1}^{\ell} (d_h - g(X_h, \omega))^2$$

Pour le résoudre, on va adopter des algorithmes ou méthodes adaptatives, qui reposent sur la génération d'une suite $(\omega_k)_{k \in \mathbb{N}}$ de poids convergeant vers le minimum de l'erreur locale $e_h(\omega)$, c'est-à-dire que ces algorithmes minimisent l'erreur locale, et non pas l'erreur totale ou moyenne.

La famille des algorithmes de descente de gradient est alors naturellement candidate. On suppose maintenant que f est dérivable. On a :

$$\begin{aligned} \forall i \in \{0, \dots, n\} : \frac{\partial e_h(\omega)}{\partial \omega_i} &= \frac{\partial}{\partial \omega_i} \left(\frac{1}{2} (S_h - d_h)^2 \right) = (S_h - d_h) \cdot \frac{\partial f(u)}{\partial u} \cdot \frac{\partial u}{\partial \omega_i} \\ &= (S_h - d_h) \cdot f'(u) \cdot \frac{\partial}{\partial \omega_i} \left(\sum_{k=0}^n \omega_k x_k^h \right) = (S_h - d_h) \cdot f'(u) \cdot x_i^h \end{aligned}$$

En posant :

$$\delta := -(d_h - S_h) \cdot f' \left(\sum_{k=0}^n \omega_k x_k^h \right) \Rightarrow \frac{\partial e_h(\omega)}{\partial \omega_i} = -\delta \cdot x_i^h$$

Le vecteur gradient de la fonction e_h s'écrit donc :

$$\nabla_{\omega} e_h(\omega) = \begin{pmatrix} \frac{\partial e_h(\omega)}{\partial \omega_0} \\ \vdots \\ \frac{\partial e_h(\omega)}{\partial \omega_n} \end{pmatrix} = -\delta \cdot \begin{pmatrix} x_0^h \\ \vdots \\ x_n^h \end{pmatrix} = -\delta X_h$$

où X_h est un vecteur d'entrée de notre ensemble de données.

Remarque : Comme le nombre ℓ est positif, on pourrait éliminer le facteur $1/2$ dans l'expression de la fonction à minimiser, mais il est préférable de le garder pour alléger l'expression du gradient.

Algorithmme : Apprentissage d'un MLP a une seule couche cachée

Algorithm 1 Apprentissage d'un MLP a une seule couche cachée

```

1: Entrées :
2: - Base de données  $\{(X_h, d_h)\}_{h=1}^{\ell}$ , avec  $X_h = (x_1^{(h)}, \dots, x_n^{(h)})$ 
3: - Fonction d'activation dérivable  $f : \mathbb{R} \rightarrow \mathbb{R}$ 
4: - Poids initiaux  $\omega = (\omega_0, \dots, \omega_n) \in \mathbb{R}^{n+1}$ 
5: - Taux d'apprentissage  $\eta > 0$ 
6: - Nombre maximal d'itérations  $N_{\max}$ 
7: Sortie : Poids optimaux  $\omega = (\omega_0, \dots, \omega_n)$ 
8:  $k \leftarrow 1$ 
9: while  $k \leq N_{\max}$  do
10:   for  $j = 0$  to  $n$  do
11:      $\Delta\omega_j \leftarrow 0$ 
12:   end for
13:   for  $h = 1$  to  $\ell$  do
14:      $x_0^{(h)} \leftarrow 1$  {Ajout du biais}
15:      $u_h \leftarrow \sum_{j=0}^n \omega_j \cdot x_j^{(h)}$ 
16:      $S_h \leftarrow f(u_h)$ 
17:      $e_h \leftarrow d_h - S_h$ 
18:      $\delta_h \leftarrow e_h \cdot f'(u_h)$ 
19:     for  $j = 0$  to  $n$  do
20:        $\Delta\omega_j \leftarrow \Delta\omega_j + \eta \cdot \delta_h \cdot x_j^{(h)}$ 
21:     end for
22:   end for
23:   for  $j = 0$  to  $n$  do
24:      $\omega_j \leftarrow \omega_j + \Delta\omega_j$ 
25:   end for
26:    $k \leftarrow k + 1$ 
27: end while
28: return  $\omega = (\omega_0, \dots, \omega_n)$ 

```

Remarques

1. Cet algorithme est inspiré de la descente de gradient classique (Cauchy 1847, Hadamard 1908), appliqué à la fonction d'erreur locale $e_h(\omega)$.
2. Le critère d'arrêt peut être choisi autrement, par exemple : $\|\nabla C(\omega)\| \leq \varepsilon$ ou $\|\omega^{(k+1)} - \omega^{(k)}\| \leq \varepsilon$.
3. η_k est le taux d'apprentissage (ou pas de descente). Il peut être :
 - fixé dès le début (descente à pas constant),
 - choisi à chaque itération (descente à pas prédéterminé),
 - ou bien déterminé par une procédure d'optimisation unidimensionnelle (descente à pas optimal).
4. En général, rien ne garantit que la diminution de l'erreur pour un point implique une diminution globale, mais cette stratégie est commune à tous les algorithmes adaptatifs.

Application à la fonction logique AND

Dans cette partie, nous considérons la fonction logique AND afin d'illustrer concrètement le processus d'apprentissage supervisé d'un neurone formel. L'objectif est d'entraîner le neurone à reproduire le comportement de cette fonction logique à partir d'un ensemble d'exemples d'apprentissage. Pour cela, deux variables d'entrée sont considérées et une sortie cible y est associée à chaque combinaison des entrées. La sortie attendue est égale à 1 lorsque les deux entrées satisfont simultanément la condition définie par la fonction AND, et égale à 0 dans les autres cas. Les paramètres du neurone, notamment les poids et le biais, sont ajustés progressivement au cours des itérations afin de minimiser l'erreur entre la sortie réelle et la sortie prédite.

```
1 import numpy as np
2 import math
3
4 def heaviside(x):
5     if x >= 0:
6         return 1
7     else:
8         return 0
9
10 def sigmoid(x):
11     return 1. / (1 + math.exp(-x))
12
13 # Initialisation
14 w = np.random.uniform(-6/math.sqrt(3), 6/math.sqrt(3), (1,2))
15 b = 0
16
17 # Donnees d'apprentissage
18 x = [[2,3], [2,0], [0,4], [5,1]]
19 y = [1,0,0,1]
20
21 max_it = 10000
22 epoch = 1
```

```

23 learning_rate = 0.01
24
25 LOSS = []
26 ERROR = []
27
28 while epoch < max_it:
29     error_s = 0
30
31     for i in range(len(x)):
32         y_hat = sigmoid(np.dot(w, np.asarray(x[i])) + b)
33
34         error = y[i] - y_hat
35         error_s += error * error * 0.5
36
37         w = w + learning_rate * error * (1-y_hat) \
38             * np.asarray(x[i], dtype=np.float32)
39
40         b = b + learning_rate * error * (1-y_hat)
41
42         print('Le vrai est', y[i],
43             'le predit est', y_hat,
44             '----- la loss est', error_s)
45
46     ERROR.append(error_s * 0.25)
47     epoch = epoch + 1
48     LOSS.append(error_s)
49
50 # Test
51 proba = sigmoid(np.dot(w, np.asarray([2,3]))) + b)
52
53 print('La proba est', proba)

```

Listing 4.5 – Implémentation de l'apprentissage supervisé d'un neurone en Python

Résultats

Après l'exécution du programme, nous obtenons les résultats suivants :

```

1 Le vrai est 1 le predit est 0.9999380477188883 ----- la loss
  est 1.919042567472866e-09
2 Le vrai est 0 le predit est 0.9917718250013459 ----- la loss
  est 0.4918056783522927
3 Le vrai est 0 le predit est 0.9985423323091421 ----- la loss
  est 0.9903490730589833
4 Le vrai est 1 le predit est 0.9999987733675362 ----- la loss
  est 0.9903490730597356
5 Le vrai est 1 le predit est 0.9999380107095245 ----- la loss
  est 1.9213360668290143e-09
6
7
8 ...
9

```

```

10
11
12
13 Le vrai est 1 le predit est 0.7352477968245934 ----- la loss
    est 0.03504686454311589
14 Le vrai est 0 le predit est 0.06730554541404474 ----- la loss
    est 0.03731188276485691
15 Le vrai est 0 le predit est 0.053577287687360725 ----- la loss
    est 0.03874714564282402
16 Le vrai est 1 le predit est 0.9981019333298936 ----- la loss
    est 0.038748946971366105
17
18 La proba est 0.7352639477373292

```

Listing 4.6 – Résultats de l'apprentissage

Les résultats obtenus montrent l'apprentissage de la fonction logique AND à partir des données fournies. Au cours des différentes itérations, les valeurs prédites évoluent progressivement afin de se rapprocher des valeurs attendues pour les différentes combinaisons d'entrées. La diminution de la valeur de la loss indique une amélioration des prédictions au cours de l'apprentissage. Enfin, pour l'entrée de test [2,3], la probabilité obtenue est de 0.7353, ce qui indique une prédiction en faveur de la classe 1.

Descente de gradient stochastique

Pour les grands ensembles de données, le calcul du gradient complet est trop coûteux. Le SGD (abréviation de *Stochastic Gradient Descent*) approxime le gradient à partir d'un seul exemple, c'est-à-dire . Au lieu de calculer un gradient complexe et de mettre à jour le point pour l'ensemble des données, nous calculons un gradient simple et mettons à jour le point pour chaque point de données individuel. La descente de gradient stochastique (SGD) sélectionne aléatoirement un point de données dans l'ensemble de données à chaque itération.

Algorithm 2 Descente de gradient stochastique

Entrées :

- Un point initial ω_0
- Un seuil d'erreur $\varepsilon > 0$

Itération : Supposons que le point ω_k soit déjà obtenu :

- (a) Calculer le gradient $\nabla e_i(\omega_k)$
- (b) Choisir un pas d'apprentissage δ_k et mettre à jour :

$$\omega_{k+1} \leftarrow \omega_k - \delta_k \nabla e_i(\omega_k)$$

où e_i est l'erreur associée à l'exemple i .

Critère d'arrêt : Arrêter lorsque

$$\|\nabla e_i(\omega_k)\| < \varepsilon$$

Cette approche introduit du bruit dans l'estimation du gradient mais permet des itérations plus rapides.

Exemple de réseau de neurones par SGD

Dans cette partie, nous utilisons un réseau de neurones entraîné avec la méthode de descente de gradient stochastique (*Stochastic Gradient Descent - SGD*) pour effectuer une tâche de classification sur le jeu de données *MNIST*. Ce jeu de données contient des images de chiffres manuscrits allant de 0 à 9.

Le code utilisé permet de charger et de préparer les données, de construire le réseau de neurones, puis de l'entraîner à l'aide de l'optimiseur SGD. Les performances du modèle sont ensuite vérifiées sur les données de test à travers une prédiction sur une image choisie aléatoirement.

```
1 import tensorflow as tf
2 from tensorflow.keras.datasets import mnist
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import random
6 from keras.models import Sequential
7 from keras.layers import Dense
8
9 # Chargement du dataset MNIST
10 (X_train, y_train), (X_test, y_test) = mnist.load_data()
11
12 # Pr traitement des donn es
13 X_train = X_train.reshape(-1, 28 * 28) / 255.0
14 X_test = X_test.reshape(-1, 28 * 28) / 255.0
15
16 y_train = tf.keras.utils.to_categorical(y_train, 10)
17 y_test = tf.keras.utils.to_categorical(y_test, 10)
18
19 # Construction du mod le
20 model = Sequential()
21 model.add(Dense(784, activation='relu', input_shape=(784,)))
22 model.add(Dense(36, activation='relu'))
23 model.add(Dense(10, activation='softmax'))
24
25 # Compilation du mod le
26 model.compile(
27     optimizer='SGD',
28     loss='categorical_crossentropy',
29     metrics=['accuracy']
30 )
31
32 # Entra nement
33 model.fit(
34     X_train,
35     y_train,
36     epochs=10,
37     batch_size=36,
38     validation_data=(X_test, y_test)
39 )
40
41 # Test sur une image al atoire
```



```

42 idx_test = random.randint(0, X_test.shape[0] - 1)
43 img_test = np.expand_dims(X_test[idx_test], axis=0)
44
45 prediction = model.predict(img_test)
46 predict_label = np.argmax(prediction)
47 true_label = np.argmax(y_test[idx_test])
48
49 print(f"Predicted label: {predict_label}")
50 print(f"True label: {true_label}")

```

Listing 4.7 – Classification des chiffres manuscrits avec un réseau de neurones et SGD

Résultats

Après l'exécution du programme, nous obtenons les résultats suivants :

```

1 Epoch 1/10
2 1667/1667 - 10s - accuracy: 0.8594 - loss: 0.5493 - val_accuracy:
   0.9160 - val_loss: 0.2936
3
4 Epoch 2/10
5 1667/1667 - 9s - accuracy: 0.9225 - loss: 0.2749 - val_accuracy:
   0.9291 - val_loss: 0.2472
6
7 Epoch 3/10
8 1667/1667 - 13s - accuracy: 0.9372 - loss: 0.2225 - val_accuracy:
   0.9423 - val_loss: 0.1968
9
10 Epoch 4/10
11 1667/1667 - 13s - accuracy: 0.9462 - loss: 0.1887 - val_accuracy:
   0.9492 - val_loss: 0.1723
12
13 Epoch 5/10
14 1667/1667 - 13s - accuracy: 0.9536 - loss: 0.1631 - val_accuracy:
   0.9551 - val_loss: 0.1532
15
16 Epoch 6/10
17 1667/1667 - 14s - accuracy: 0.9594 - loss: 0.1434 - val_accuracy:
   0.9593 - val_loss: 0.1368
18
19 Epoch 7/10
20 1667/1667 - 14s - accuracy: 0.9642 - loss: 0.1276 - val_accuracy:
   0.9631 - val_loss: 0.1255
21
22 Epoch 8/10
23 1667/1667 - 14s - accuracy: 0.9679 - loss: 0.1145 - val_accuracy:
   0.9656 - val_loss: 0.1186
24
25 Epoch 9/10
26 1667/1667 - 15s - accuracy: 0.9711 - loss: 0.1036 - val_accuracy:
   0.9674 - val_loss: 0.1114
27

```

```

28 Epoch 10/10
29 1667/1667 - 14s - accuracy: 0.9734 - loss: 0.0947 - val_accuracy:
    0.9701 - val_loss: 0.1023
30
31 1/1 - 0s 181ms/step
32
33 Predicted label: 6
34 True label: 6

```

Listing 4.8 – Résultats de l’entraînement du réseau de neurones avec SGD

La précision obtenue par le modèle sur l’ensemble de test est de 97.01 %. Cela signifie que le modèle a correctement classifié environ 97 images sur 100. On remarque également que la précision augmente progressivement au cours des 10 époques, tandis que la valeur de la perte diminue. Enfin, le modèle a correctement prédit l’image testée, en identifiant le chiffre 6, qui correspond à la vraie étiquette.

Adam

Adam est l’un des algorithmes les plus récents et les plus efficaces pour l’optimisation par descente de gradient. Sa particularité est qu’il calcule (m_k, v_k) , qui sont des estimations adaptatives des moments.

Les règles de mise à jour sont les suivantes :

$$\forall i, \quad (m_{k+1})_i \leftarrow \beta_1(m_k)_i + (1 - \beta_1)(\nabla E(\omega_k))_i$$

$$\forall i, \quad (v_{k+1})_i \leftarrow \beta_2(v_k)_i + (1 - \beta_2)(\nabla E(\omega_k))_i^2$$

$$\forall i, \quad (\theta_{k+1})_i \leftarrow (\theta_k)_i - \alpha \frac{\sqrt{1 - \beta_2^{k+1}}}{1 - \beta_1^{k+1}} \frac{(m_{k+1})_i}{\sqrt{(v_{k+1})_i} + \varepsilon}$$

avec

$$\alpha, \varepsilon > 0, \quad \beta_1, \beta_2 \in (0, 1).$$

Ici, m_k représente le premier moment du gradient (la moyenne), et v_k représente son second moment (la variance non centrée). Le paramètre ε assure la stabilité numérique.

Les paramètres β_1 et β_2 sont utilisés pour calculer les moyennes mobiles exponentielles respectives du premier et du second moment. Cette technique de lissage attribue un poids décroissant aux gradients passés, permettant ainsi d’obtenir une estimation plus stable et adaptative des moments.

Ces algorithmes d’optimisation sont au cœur de l’entraînement des réseaux de neurones. Leur choix et leur paramétrage influencent directement la qualité des résultats obtenus.

Les réseaux de neurones convolutifs

5.1 Introduction sur les Convolutional Neural Networks

Les Convolutional Neural Networks (CNN) sont une architecture spécialement conçue pour traiter des données à structure spatiale, comme les images. Grâce à leur structure, ils extraient automatiquement les caractéristiques clés d'une image pour réaliser diverses tâches de reconnaissance et de classification.

Aujourd'hui incontournables en Computer Vision, les CNN sont utilisés pour la classification, la détection d'objets, la reconnaissance faciale ou l'imagerie médicale. Leur force repose sur leur capacité à prendre en compte la position relative et les relations entre les pixels.

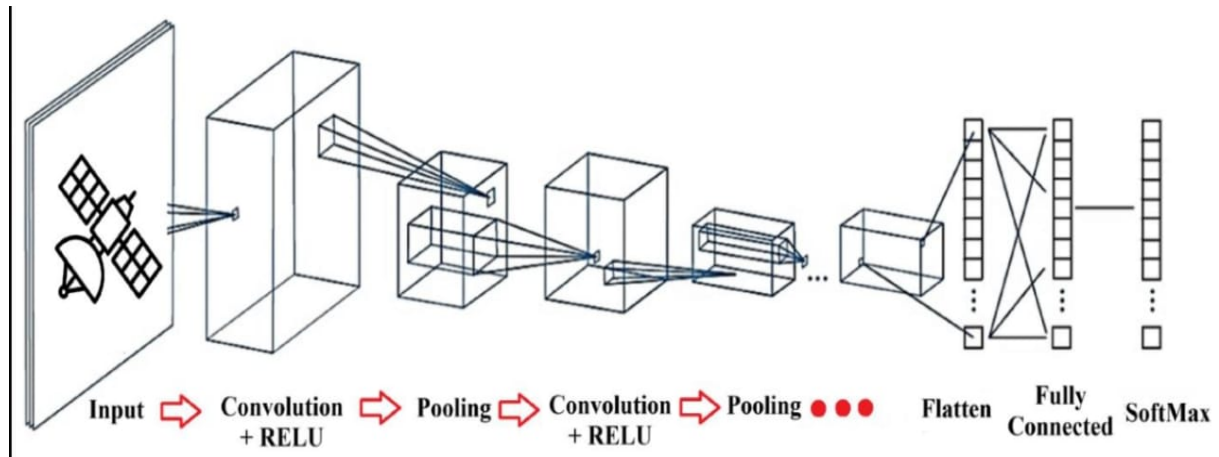
Pour comprendre leur fonctionnement, il faut rappeler qu'une image numérique est une matrice de pixels (une seule matrice pour le niveau de gris, trois matrices RGB pour la couleur). Chaque pixel contient une valeur d'intensité ou de couleur.

Pourquoi ne pas utiliser un Deep Neural Network (DNN) classique ? Bien qu'un DNN puisse recevoir une image en entrée, l'aplatir en un simple vecteur génère un nombre massif de paramètres. Cela entraîne un coût de calcul élevé, une forte consommation mémoire et un risque important de surapprentissage (overfitting).

De plus, un DNN classique ne prend pas en compte la structure spatiale de l'image et perd l'information de proximité entre les pixels. Les CNN résolvent ce problème grâce aux couches de convolution, qui analysent l'image localement.

Ainsi, le CNN apprend une hiérarchie de caractéristiques : d'abord des éléments simples (bords, lignes), puis des formes plus complexes, jusqu'à identifier l'objet entier. C'est cette extraction automatique combinée au respect de la structure spatiale qui fait l'efficacité des CNN.

5.2 Architecture générale d'un CNN



L'architecture générale d'un réseau de neurones convolutif (CNN) est composée de plusieurs couches qui permettent de transformer progressivement une image en une représentation adaptée à la classification. Comme illustré dans la figure, le traitement commence par une image d'entrée (Input), qui est ensuite transmise à plusieurs couches successives.

La première étape est la convolution, qui consiste à appliquer des filtres sur l'image afin d'extraire des caractéristiques importantes, telles que les bords, les lignes et les textures. La fonction d'activation ReLU (Rectified Linear Unit) est généralement appliquée après la convolution afin d'introduire de la non-linéarité dans le réseau.

Ensuite, une couche de Pooling est utilisée pour réduire la taille des cartes de caractéristiques tout en conservant les informations les plus importantes. Les opérations de convolution, ReLU et pooling peuvent être répétées plusieurs fois afin d'extraire progressivement des caractéristiques de plus en plus complexes.

Après ces différentes étapes, les cartes de caractéristiques obtenues sont transformées en un vecteur à l'aide de l'opération Flatten. Ce vecteur est ensuite transmis aux couches Fully Connected, qui combinent les caractéristiques extraites afin de prendre une décision de classification.

Enfin, la couche SoftMax fournit les probabilités associées aux différentes classes. La classe ayant la probabilité la plus élevée correspond à la prédiction finale du réseau.

Nous abordons à présent les concepts de base.

5.2.1 Convolution

La convolution constitue l'une des principales opérations d'un CNN. Elle consiste à faire glisser un petit filtre, appelé kernel, sur l'image afin d'extraire certaines caractéristiques locales. À chaque position, le filtre effectue des multiplications entre ses valeurs et celles des pixels correspondants, puis calcule leur somme. Le résultat obtenu forme une nouvelle matrice appelée carte de caractéristiques (feature map).

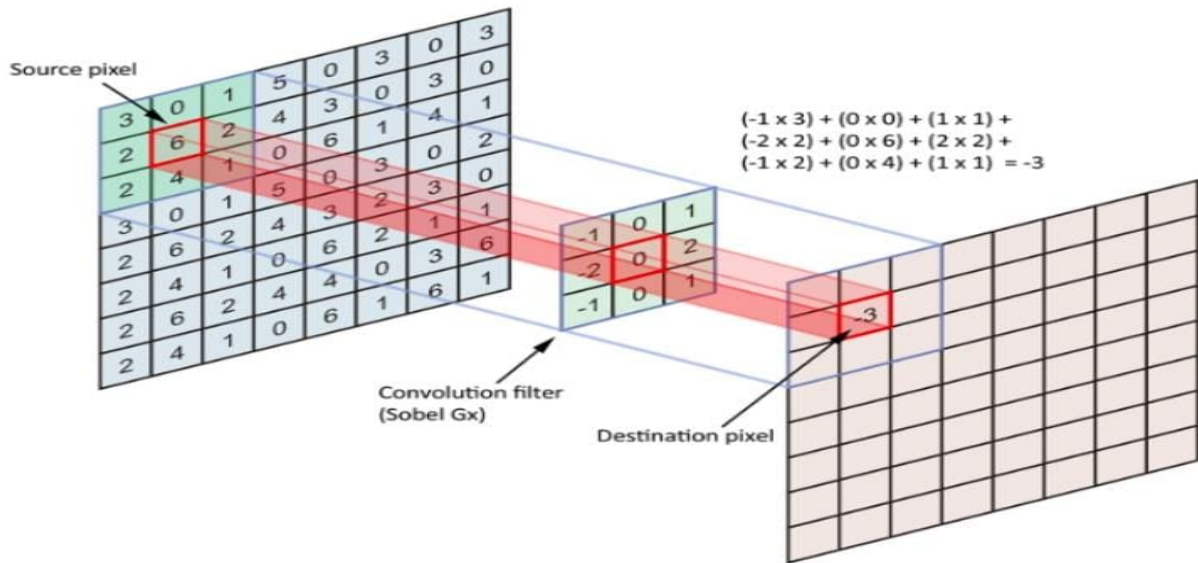


FIGURE 5.1 – Exemple d’une opération de convolution 3×3 à l’aide d’un filtre de Sobel G_x .

Comme l’illustre l’exemple de la Figure 5.1, le filtre appliqué (ici un filtre de Sobel G_x de taille 3×3) effectue le calcul suivant :

$$\text{Valeur} = (-1 \times 3) + (0 \times 0) + (1 \times 1) + (-2 \times 2) + (0 \times 6) + (2 \times 2) + (-1 \times 2) + (0 \times 4) + (1 \times 1) = -3$$

Ce résultat donne la valeur -3 pour le pixel de destination correspondant dans la feature map.

Il est important de souligner qu’une seule couche de convolution utilise généralement plusieurs filtres en parallèle. Chaque filtre possède son propre kernel (avec des poids différents), ce qui lui permet de se spécialiser dans la détection d’un type précis de caractéristique (par exemple, un filtre pour les contours horizontaux, un autre pour les contours verticaux ou les gradients de couleur). L’empilement des cartes de caractéristiques générées par l’ensemble de ces filtres constitue le volume de sortie de la couche convolutive.

5.2.2 Pooling

La couche de pooling (ou sous-échantillonnage) intervient généralement après les couches de convolution. Son objectif principal est de réduire la dimension spatiale (largeur et hauteur) des cartes de caractéristiques (feature maps), ce qui permet de diminuer la complexité calculatoire, d’alléger le nombre de paramètres et de réduire le risque de surapprentissage (overfitting). Il est important de noter que l’opération de pooling s’applique de manière indépendante à chaque carte de caractéristiques (feature map) du volume d’entrée, préservant ainsi le nombre total de canaux (profondeur).

Le principe repose sur le glissement d’une fenêtre (par exemple 2×2) sur la matrice d’entrée pour appliquer une fonction d’agrégation. Bien qu’il existe plusieurs types de

fonctions d'agrégation (comme la valeur maximale, la moyenne, le minimum ou la norme L_2), deux méthodes principales se distinguent, comme l'illustre la Figure 5.2 :



FIGURE 5.2 – Principe du Max Pooling et de l’Average Pooling.

Max Pooling : Consiste à conserver uniquement la valeur maximale au sein de chaque région. Dans l’exemple illustré (région verte), pour les valeurs $\{29, 15, 0, 100\}$, le résultat est $\max(29, 15, 0, 100) = 100$. C’est la fonction d’agrégation la plus largement utilisée en pratique, car elle s’avère particulièrement efficace pour capturer les caractéristiques saillantes et les contours tout en offrant une meilleure invariance aux petites transformations géométriques.

Average Pooling : Calcule la valeur moyenne des pixels de la fenêtre. Pour la même zone verte, le calcul donne $\frac{31+15+0+100}{4} = 36$. Cette méthode est plutôt privilégiée lorsque l’on souhaite conserver une information globale plus lissée sur l’ensemble de la région (par exemple dans certaines couches finales comme le Global Average Pooling).

5.2.3 Flatten

L’étape d’aplatissement (Flattening) constitue la transition essentielle entre la partie d’extraction de caractéristiques (Feature Extraction) du réseau et la partie de classification. Après avoir appliqué une succession de couches de convolution et de pooling, les données se trouvent sous la forme d’un ensemble de cartes de caractéristiques bidimensionnelles ou tridimensionnelles (volume de cartes de caractéristiques).

Comme le montre la Figure 5.3, l’opération de Flattening consiste à transformer l’ensemble de ces matrices multidimensionnelles issues de la dernière couche de pooling en un seul vecteur unidimensionnel (une colonne continue de valeurs). Cette conversion est indispensable car les couches entièrement connectées (Fully Connected Layers ou ANN classique) situées en fin de réseau nécessitent des données d’entrée sous forme de vecteur à une seule dimension.

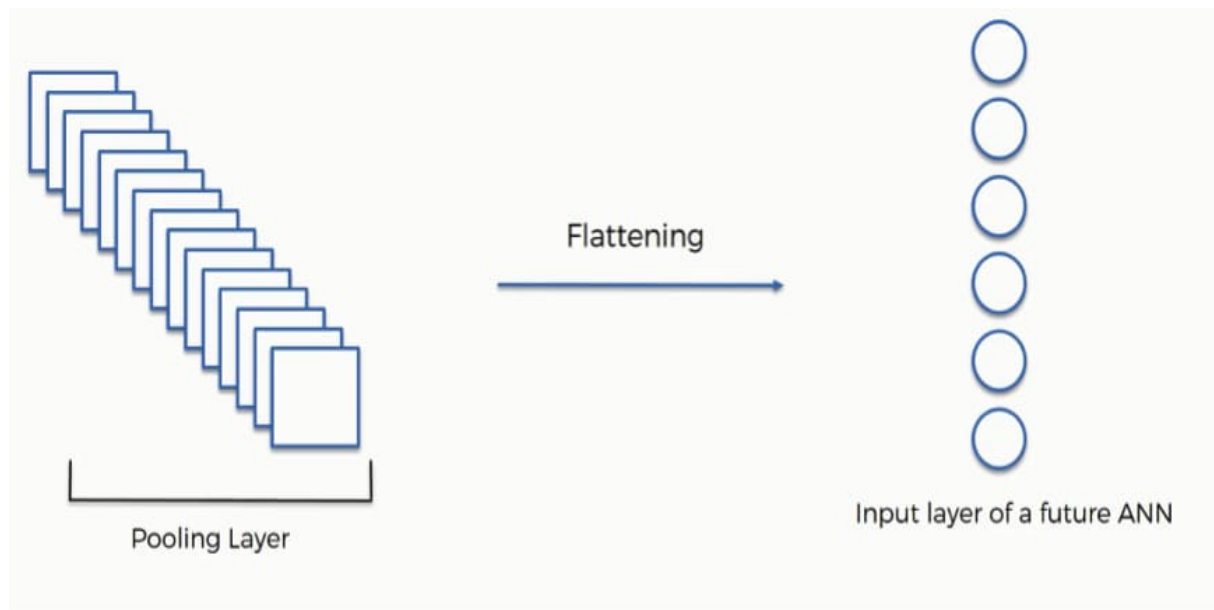


FIGURE 5.3 – Principe de l'opération d'aplatissement (Flattening).

5.2.4 Fully Connected

La couche entièrement connectée (Fully Connected Layer, notée FC ou couche Dense) constitue l'étape finale de l'architecture d'un CNN. Elle intervient immédiatement après l'étape d'aplatissement (Flattening).

Alors que les couches de convolution et de pooling ont pour rôle d'extraire et de structurer les caractéristiques visuelles de l'image, la couche Fully Connected assure la phase de classification.

Son fonctionnement repose exactement sur les mêmes principes que ceux du réseau de neurones dense (DNN) présentés dans le Chapitre 1. Les éléments du vecteur issu du Flattening sont transmis aux neurones de la couche dense, où chaque connexion applique ses poids et biais, suivis d'une fonction d'activation.

5.2.5 SoftMax

Enfin, dans le cas d'une classification multiclasse, la fonction SoftMax transforme les sorties du réseau en probabilités associées aux différentes classes. La classe ayant la probabilité la plus élevée est alors considérée comme la prédiction du modèle.

5.3 Entraînement du réseau de neurones convolutifs

Une fois l'architecture du CNN définie (couches de convolution, de pooling, flattening et couches entièrement connectées), le réseau doit suivre une phase d'apprentissage (ou entraînement). L'objectif de cette étape est d'ajuster progressivement les paramètres optimaux du réseau (principalement les poids w et les biais b des filtres et des couches denses) afin de minimiser les erreurs de prédiction sur les données d'entraînement.

L'entraînement d'un CNN repose sur un processus itératif structuré en trois grandes étapes clés :

5.3.1 La propagation avant (Forward Propagation) :

L'image d'entrée passe à travers l'ensemble des couches du réseau (convolution, pooling, FC, activation). Le modèle génère une prédiction finale sous forme de probabilités (via Softmax).

5.3.2 Le calcul de la perte (Loss Calculation) :

La prédiction du modèle est comparée à la réalité (ground truth ou étiquette réelle) à l'aide d'une fonction de perte (Loss Function), telle que l'entropie croisée (Categorical Cross-Entropy pour le multiclasse ou Binary Cross-Entropy pour le binaire). La fonction de perte mesure l'écart ou l'erreur commise par le réseau.

5.3.3 La rétropropagation et l'optimisation (Backpropagation Optimization) :

L'erreur calculée est rétropropagée à travers le réseau, du dernier niveau vers les premières couches, en calculant le gradient de la fonction de perte par rapport à chaque poids. Un algorithme d'optimisation (tel que SGD, Adam ou RMSprop) met ensuite à jour les poids pour réduire l'erreur lors des itérations suivantes (Epochs).

5.4 Application des modèles ANN et CNN

Après avoir présenté les principes théoriques des réseaux de neurones convolutifs, nous passons dans cette partie à leur mise en œuvre pratique. Nous mettons en œuvre deux modèles de classification, ANN et CNN, afin de comparer leurs performances sur le dataset CIFAR-10. Après le chargement et la normalisation des images, un modèle ANN basé sur des couches entièrement connectées est développé comme modèle de référence. Ensuite, un modèle CNN utilisant des couches de convolution et de MaxPooling est construit afin d'extraire automatiquement les caractéristiques spatiales des images. Les deux modèles sont entraînés et évalués sur les mêmes données afin de comparer leurs performances et de mettre en évidence l'intérêt des CNN pour la classification d'images.

```
1
2 import tensorflow as tf
3 from tensorflow.keras import datasets, layers, models
4 import matplotlib.pyplot as plt
5 import numpy as np
6 from sklearn.metrics import confusion_matrix, classification_report
7
8 (X_train, y_train), (X_test, y_test) = datasets.cifar10.load_data()
9
10 y_train = y_train.reshape(-1,)
11 y_test = y_test.reshape(-1,)
12
13 classes = ["airplane", "automobile", "bird", "cat", "deer", "dog", "frog",
14            "horse", "ship", "truck"]
15
16 def plot_sample(X, y, index):
17     plt.figure(figsize=(3, 3))
18     plt.imshow(X[index])
19     plt.title(classes[y[index]])
```



```

19     plt.axis("off")
20     plt.show()
21
22 plot_sample(X_train, y_train, 0)
23 plot_sample(X_train, y_train, 1)
24
25 X_train = X_train / 255.0
26 X_test = X_test / 255.0
27
28
29 ann = models.Sequential([
30     layers.Flatten(input_shape=(32,32,3)),
31     layers.Dense(3000, activation='relu'),
32     layers.Dense(1000, activation='relu'),
33     layers.Dense(10, activation='softmax')
34 ])
35
36 ann.compile(optimizer='SGD',
37             loss='sparse_categorical_crossentropy',
38             metrics=['accuracy'])
39
40 ann.fit(X_train, y_train, epochs=5)
41
42 y_pred = ann.predict(X_test)
43 y_pred_classes = [np.argmax(element) for element in y_pred]
44
45 print("Classification Report: \n", classification_report(y_test,
46     y_pred_classes))
47
48 cnn = models.Sequential([
49     layers.Conv2D(filters=32, kernel_size=(3, 3), activation='relu',
50     , input_shape=(32, 32, 3)),
51     layers.MaxPooling2D((2, 2)),
52
53     layers.Conv2D(filters=64, kernel_size=(3, 3), activation='relu',
54     ),
55     layers.MaxPooling2D((2, 2)),
56
57     layers.Flatten(),
58     layers.Dense(64, activation='relu'),
59     layers.Dense(10, activation='softmax')
60 ])
61
62 cnn.compile(optimizer='adam',
63             loss='sparse_categorical_crossentropy',
64             metrics=['accuracy'])
65
66 history = cnn.fit(
67     X_train,
68     y_train,

```

```

67     epochs=10,
68     validation_split=0.2
69 )
70
71 cnn.evaluate(X_test,y_test)
72
73 y_pred = cnn.predict(X_test)
74 y_pred[:5]
75
76 y_classes = [np.argmax(element) for element in y_pred]
77 y_classes[:5]
78
79 print("True classes:", y_test[:5])
80 print("Predicted classes:", y_classes[:5])
81
82 plot_sample(X_test, y_test,3)

```

Listing 5.1 – Implémentation et comparaison des modèles ANN et CNN pour la classification d’images avec CIFAR-10

Résultats

TABLE 5.1 – Résultats obtenus lors de l’entraînement et de l’évaluation des modèles ANN et CNN

Modèle	Accuracy entraînement	Accuracy validation	Accuracy test	Loss test
ANN	49.64%	—	—	—
CNN	80.98%	68.36%	68.04%	1.0220

Les résultats obtenus montrent que le modèle CNN présente de meilleures performances pour la classification des images du dataset CIFAR-10. Au cours de l’entraînement, le CNN atteint une accuracy de 80,98%, tandis que son accuracy sur les données de validation est de 68,36%. Lors de l’évaluation sur les données de test, le modèle obtient une accuracy de 68,04% avec une loss de 1,0220. En comparaison, l’ANN atteint une accuracy d’entraînement de 49,64%. Ces résultats montrent l’intérêt des couches de convolution et de MaxPooling, qui permettent au CNN d’extraire automatiquement les caractéristiques spatiales des images et d’obtenir ainsi de meilleures performances que l’ANN pour cette tâche de classification.

Conclusion et Perspectives

Synthèse des travaux et résultats

Dans ce rapport, nous avons mené une étude approfondie des réseaux de neurones artificiels (RNAs), combinant fondements théoriques, aspects algorithmiques et architectures spécialisées.

Après avoir posé le cadre et les objectifs de l'étude, nous avons introduit les bases mathématiques et conceptuelles des RNAs. L'analyse du neurone formel (McCulloch et Pitts) et du perceptron simple, à travers les portes logiques (AND, OR, NOT, XOR), a permis d'illustrer la puissance ainsi que les limites fondamentales des modèles linéaires. La résolution du problème non linéairement séparable du XOR a notamment souligné la nécessité d'introduire des couches cachées. Cette analyse de structure s'est étendue aux réseaux à propagation avant (FFNN), aux réseaux récurrents (RNN) et aux perceptrons multicouches (MLP), tout en évaluant le rôle clé des fonctions d'activation (Sigmoid, ReLU, Tanh) sur la dynamique de l'apprentissage.

Sur le plan de la théorie fondamentale, nous avons exploré la capacité d'approximation universelle des RNAs. L'analyse rigoureuse du théorème de Cybenko a établi qu'un réseau à une seule couche cachée, muni d'une fonction d'activation sigmoïdale, peut approximer toute fonction continue sur un ensemble compact. Les travaux de Hornik ont ensuite étendu cette garantie théorique à des fonctions d'activation arbitraires et non bornées, tout en établissant les implications théoriques et limites pratiques de ces théorèmes.

La question de l'apprentissage a été abordée sous l'angle de la minimisation de fonctions de coût (régression et classification). Nous avons étudié la formulation de l'optimisation supervisée et comparé plusieurs algorithmes d'optimisation, mettant en évidence les mécanismes permettant d'ajuster les poids du réseau pour réduire l'erreur empirique.

Enfin, une attention particulière a été accordée aux réseaux de neurones convolutifs (CNN). Nous avons détaillé leur architecture générale, construite autour des opérations de convolution (extraction de motifs locaux), de pooling (réduction dimensionnelle et invariance spatiale), de flattening, ainsi que des couches entièrement connectées (Fully Connected). La dynamique complète de l'entraînement a été analysée : de la propagation avant (Forward Propagation) au calcul de la perte, jusqu'à la rétropropagation (Backpropagation) pour l'ajustement des paramètres. Enfin, l'utilisation de la fonction SoftMax en fin de réseau a permis de transformer les scores bruts (logits) en une distribution de probabilités normalisée, identifiant la classe la plus probable comme prédiction finale.

Les applications numériques et expérimentations pratiques réalisées en Python ont permis de valider ces concepts théoriques, d'analyser l'impact de l'architecture et de comparer les performances des modèles ANN et CNN.

Limites et améliorations possibles

Malgré la portée des résultats obtenus, plusieurs limites doivent être soulignées :

- **Garanties théoriques vs réalités pratiques :** Bien que le théorème d'approximation universelle garantisse l'existence d'un réseau capable d'approximer une fonction donnée, il ne fournit aucune indication sur le nombre de neurones requis ni sur la faisabilité de trouver ces poids via les algorithmes d'optimisation habituels.
- **Défis d'optimisation :** La convergence vers un minimum global reste difficile dans des paysages de perte non convexes, exposant les modèles aux risques d'évanouissement ou d'explosion du gradient.
- **Complexité calculatoire des CNN :** L'entraînement des architectures convolutives profondes exige un volume important de données étiquetées et des ressources de calcul soutenues pour éviter le surapprentissage (overfitting).

Perspectives pour des travaux futurs

Plusieurs axes de recherche et d'expérimentation permettraient de prolonger ce travail :

1. **Algorithmes d'optimisation avancés et régularisation :** Approfondir l'étude d'optimiseurs adaptatifs modernes tels qu'Adam, RMSProp ou AdaGrad, et intégrer des techniques de régularisation poussées (L_1/L_2 , Dropout, Batch Normalization) pour améliorer la généralisation des modèles.
2. **Architectures profondes et modernes :** Étendre la comparaison expérimentale aux réseaux résiduels (ResNet), aux réseaux de neurones récurrents avancés (LSTM, GRU) et aux architectures à base d'attention (Transformers), en analysant le rôle de la profondeur dans la capacité représentative.
3. **Extension mathématique aux espaces fonctionnels complexes :** Explorer l'approximation universelle dans des cadres plus généraux, tels que les fonctions discontinues, les distributions ou la géométrie d'apprentissage sur graphes (Graph Neural Networks).

Bibliographie

- [1] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4) :303–314, 1989.
- [2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. <https://www.deeplearningbook.org>.
- [3] Kurt Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2) :251–257, 1991.
- [4] Diederik P. Kingma and Jimmy Ba. Adam : A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. <https://arxiv.org/abs/1412.6980>.
- [5] Moshe Leshno, Vladimir Y. Lin, Allan Pinkus, and Shimon Schocken. Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural Networks*, 6(6) :861–867, 1993.
- [6] Allan Pinkus. Approximation theory of the mlp model in neural networks. *Acta Numerica*, 8 :143–195, 1999.
- [7] Sebastian Ruder. An overview of gradient descent optimization algorithms. <https://arxiv.org/abs/1609.04747>, 2016.
- [8] Shai Shalev-Shwartz and Shai Ben-David. *Understanding Machine Learning : From Theory to Algorithms*. Cambridge University Press, 2014.
- [9] Matus Telgarsky. Benefits of depth in neural networks. In *Conference on Learning Theory (COLT)*, 2016.
- [10] Vladimir N. Vapnik. *Statistical Learning Theory*. Wiley, 1998.
- [11] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. In *International Conference on Learning Representations (ICLR)*, 2017. <https://arxiv.org/abs/1611.03530>.